

LumiNovels: A Hybrid Approach For Light Novel Recommendations

Year 3 - CS310 Final Report

MD Tarek Islam [1]
ID: 2100018

With the assistance of

Dr. Ayse Saliha Sunar [2]
and
Dr. Florin Ciucu [3]

[1]Undergraduate Student, Department of Computer Science
[2]Supervisor - Teaching Fellow, Department of Computer Science
[3]Second Assessor - Associate Professor, Department of Computer Science

Abstract

This study introduces an innovative recommendation system explicitly tailored for light novels, addressing inherent challenges such as popularity bias and the cold start problem. Harnessing an online API meticulously curates a robust dataset encompassing crucial attributes like titles, authors, genres, synopses, and alternative names.

At the heart of the system lies a sophisticated neural network architecture meticulously trained to classify light novel data accurately. Content-based filtering techniques augment this framework, leveraging advanced natural language processing methodologies to thoroughly analyze synopses and extract salient features.

This hybrid approach amalgamates collaborative and content-based filtering strengths, poised to furnish users with highly personalized and pertinent recommendations. By intricately processing synopses using natural language processing techniques, the system aims to capture nuanced user preferences, enhancing the overall user experience within the light novel domain.

Keywords: Light Novels, Neural Networks, Content-based Filtering, Hybrid System.

Contents

1 Introduction . . . 5
1.1 Introduction to Recommendation Systems . . . 5
1.2 The Role of Machine Learning in Recommendation Systems . . . 5
1.3 Common Issues Faced by Machine Learning-Based Recommender Systems . . . 6
1.3.1 Cold Start Problem . . . 6
1.3.2 Data Sparsity . . . 6
1.3.3 Popularity Bias . . . 7
1.3.4 Grey Sheep Problem . . . 7
1.3.5 Lack of Online Datasets and Limited Content Analysis . . . 7
1.4 Introduction to Light Novels and Their Relevance . . . 7
1.5 Motivation behind Developing LumiNovels . . . 8
1.6 Anibrain.ai . . . 9
2 Background . . . 10
2.1 Literature Review . . . 10
2.2 Neural Network Classification . . . 17
2.3 Content-Based Filtering . . . 18
2.4 Hybridisation . . . 19
2.5 Requirements . . . 19
3 Methodology . . . 21
3.1 System Diagram . . . 21
3.2 MyAnimeList API Dataset . . . 23
3.3 Data Preprocessing . . . 25
3.4 Dataset Analysis . . . 25
3.5 Natural Language Processing . . . 29
3.6 Genre Merging . . . 30
3.7 Text Vectorisation . . . 31
4 Development . . . 33
4.1 Neural Network Configuration . . . 33
4.2 Content Based Filtering Approach: Cosine Similarity . . . 35
5 Results and Evaluation . . . 36
5.1 Initial Evaluation Metrics . . . 36
5.2 Comparison of Different Model Configurations . . . 38
5.3 Changing Epochs, Layer Size, and Learning Rate . . . 39

5.4 Exploration of Evaluation Techniques like ROC and AUC . . . 40
5.5 Cross-validation vs. Train-test Split . . . 40
5.6 Genre Reduction Using K-means++ . . . 42
6 Conclusions and Future Work . . . 47
6.1 Summary of Main Results and Achievements . . . 47
6.2 Critique of Achievements concerning Project Goals . . . 47
6.3 Identification of Difficulties, Limitations, and Lessons Learned . . . 47
6.4 Suggestions for Further Work and Extensions to the Project . . . 48
7 Author's Assessment of the Project . . . 49
8 Project Management . . . 49
8.1 Project Management Practices . . . 49
8.2 Objectives and Goals . . . 50
8.3 Supervisor Meetings . . . 51
8.4 Challenges faced . . . 51
8.5 Timetable . . . 52
9 Ethical and Legal Considerations . . . 53
10 Acknowledgements . . . 53
References . . . 54
A Genre Rating Graph . . . 60
B Genre Distribution Graph . . . 61
C Gantt Chart . . . 62

1. Introduction

1.1. Introduction to Recommendation Systems

Recommendation systems have become an integral part of modern-day online platforms. They facilitate personalised user experiences by suggesting items of interest based on past behaviour, preferences, and contextual information. These systems are crucial in enhancing user engagement, increasing customer satisfaction, and driving revenue for businesses across various domains, such as e-commerce [1], streaming services, and content platforms.

1.2. The Role of Machine Learning in Recommendation Systems

Machine learning is the backbone of recommendation systems, empowering them to analyse vast amounts of data and extract meaningful patterns to make accurate predictions. By leveraging machine learning algorithms, recommendation systems can adapt to evolving user preferences, improve recommendation quality, and handle complex decision-making tasks more effectively [2].

Machine learning techniques enable recommendation systems to learn from user interactions and feedback, generating personalised recommendations that align with individual preferences and interests. This adaptive and data-driven approach enhances user satisfaction and engagement, ultimately driving business success and customer loyalty.

Commonly used machine learning algorithms for recommendation systems include collaborative filtering, content-based filtering, and hybrid approaches. Collaborative filtering techniques analyse user-item interaction data to identify similarities between users or items, while content-based filtering methods use item features and user profiles to make recommendations. Hybrid approaches combine the strengths of both collaborative and content-based filtering to enhance recommendation accuracy and coverage.

Recommender systems work best via hybrid approaches utilising artificial neural networks. These systems leverage machine learning techniques to integrate content-based and collaborative features, enhancing accuracy and personalised recommendations.

Content-based features in recommender systems are vital because they allow the system to recommend items similar to ones liked by the user in the past. The system can make personalised recommendations based on these similarities by analysing the relationship between items and user preferences. On the other hand, collaborative features evaluate an item's relevance for a user based on the opinions of a community or cluster of users with similar interests. This approach leverages the idea that users with similar preferences prefer identical items [3].

Integrating content-based and collaborative features in a hybrid recommender system using machine learning techniques allows for a more comprehensive analysis of user preferences and item characteristics. Hybridisation leads to more accurate predictions and personalised recommendations for the users.

Machine learning enables the system to process and learn from large amounts of data. At the same time, integrating content-based and collaborative features allows for a more holistic and accurate understanding of user preferences, leading to improved recommendations.

1.3. Common Issues Faced by Machine Learning-Based Recommender Systems

Traditional machine learning-based recommender systems often encounter several challenges that can hinder their accuracy and effectiveness in providing personalised recommendations. These challenges include:

1.3.1. Cold Start Problem

The cold start problem is the difficulty of making accurate recommendations for new users or items with limited historical interaction data. Traditional recommender systems need more relevant user preferences or item characteristics to generate meaningful recommendations in such cases [4]. This issue is particularly prevalent in newly launched platforms or niche content domains where user engagement data is sparse.

1.3.2. Data Sparsity

Data sparsity arises when more than the available interaction data between users and items is needed to build robust recommendation models. In domains with a vast number of items and a diverse user base, it is common for recommender systems to encounter sparse user-item interaction matrices, where most users have only interacted with a small subset of items [5]. Some entries in the interaction matrix may also exist but need more data, further exacerbating the sparsity issue. This lack of data makes feature extraction more complex and can hinder the ability of recommendation algorithms to capture user preferences and item similarities accurately. As a result, traditional algorithms may need help to generate meaningful recommendations, leading to suboptimal user experiences.

1.3.3. Popularity Bias

Popularity bias occurs when recommender systems disproportionately recommend popular or trending items at the expense of niche or less mainstream content. This bias can arise from the inherent structure of recommendation algorithms, which often prioritize items with higher overall popularity or user engagement metrics. As a result, users may be exposed to limited content options, reducing the diversity and novelty of recommendations and potentially leading to user dissatisfaction. Users with less interest in popular items experience the poorest recommendation accuracy [6].

1.3.4. Grey Sheep Problem

The grey sheep problem refers to the challenge of accurately recommending items for users with eclectic or unconventional preferences that do not align with most users [7]. Traditional recommender systems may struggle to identify and cater to these "grey sheep" users as their preferences deviate significantly from the mainstream. As a result, these users may receive recommendations that do not reflect their unique tastes and preferences, leading to decreased user satisfaction and engagement.

1.3.5. Lack of Online Datasets and Limited Content Analysis

Another significant challenge machine learning-based recommender systems face is the scarcity of online datasets tailored explicitly to certain content domains, such as light novels [8]. The limited availability of comprehensive datasets containing user-item interaction data hampers the training and evaluation of recommendation algorithms, thereby hindering their effectiveness in generating accurate and relevant recommendations. Additionally, the need for extensive content analysis tools tailored to the characteristics of light novels further complicates the recommendation process, limiting the system's ability to extract meaningful features and insights from the available data.

Addressing these challenges is crucial for improving the performance and usability of machine learning-based recommender systems across various domains. By developing innovative algorithms and incorporating advanced techniques to mitigate the cold start problem, alleviate data sparsity, reduce popularity bias, and address the grey sheep problem, recommender systems can deliver more accurate, diverse, and personalized recommendations that better meet users' needs and preferences.

1.4. Introduction to Light Novels and Their Relevance

Light novels, a popular form of Japanese literature, have garnered substantial domestic and international attention [9]. These novels typically span around 200 pages, with approximately 50,000 words,

akin to traditional novels. While they contain fewer illustrations than manga - Japanese comics - light novels often feature a handful of illustrations scattered throughout the text to complement the narrative.

Despite being less popular than manga, light novels hold significant cultural importance as they are source material for many manga and anime adaptations [10]. Their concise chapters, narrative illustrations, and simplified grammar, tailored to young adult readers, contribute to their appeal to a diverse audience. Light novels frequently blend fantasy, science fiction, and romance elements, resulting in multifaceted narratives that resonate with readers.

Given their broad appeal and cultural significance, light novels have emerged as a compelling subject for recommendation systems. Their unique storytelling style and wide-ranging themes make them an ideal candidate for personalized and contextually relevant recommendations, enhancing the reading experience for enthusiasts of Japanese literature and beyond.

1.5. Motivation behind Developing LumiNovels

The motivation behind developing LumiNovels stems from the need to address the unique challenges of recommending light novels. In recent years, there has been a significant surge in the popularity of light novels within Japan and international audiences [10]. This growing market of light novels outside of Japan highlights the need for specialized recommendation systems that can effectively cater to the preferences of a diverse global audience.

While existing recommendation systems have demonstrated effectiveness in various domains, their applicability to niche content like light novels is limited. Generic recommendation algorithms may need help capturing the nuanced characteristics and preferences associated with light novels, resulting in suboptimal recommendations for users interested in this genre.

LumiNovels aims to bridge this gap by offering a specialized recommendation system tailored to light novels' unique characteristics. By leveraging advanced machine learning techniques and domain-specific features, LumiNovels seeks to provide users with personalized and contextually relevant recommendations that align with their tastes and preferences.

Furthermore, LumiNovels aims to promote diversity and exploration within the light novel genre by enabling users to discover new titles and authors they may not have encountered otherwise. By recommending titles similar to what users have enjoyed in the past and introducing them to new and diverse content, LumiNovels aims to enrich the reading experience and foster a sense of discovery and exploration among its users.

The growing market of light novels outside of Japan and the need for a specialized recommender system to cater to the preferences of a diverse global audience drives the development of LumiNovels.

By offering personalized recommendations and promoting diversity within the genre, LumiNovels aims to enhance the reading experience for light novel enthusiasts worldwide.

1.6. Anibrain.ai

Current recommendation systems are tailored to specific content niches such as anime and manga, mainly underserving the domain of light novels. Anibrain.ai[11] is the only existing light novel recommender that operates as a web application employing a hybrid approach, combining collaborative user and content-based filtering methodologies to generate suggestions. Its algorithm accommodates diverse user preferences by offering options for requesting entirely random light novels or those similar to a specified input.

Anibrain.ai incorporates story and genre similarity filters to enhance recommendation precision. However, its requirement for users to input a light novel as a reference may present a barrier to entry for newcomers to the genre. Notably, the system prioritizes factors beyond popularity when generating recommendations, focusing instead on contextual features to deliver more personalized suggestions.

2. Background

2.1. Literature Review

Table 1 presents a comprehensive and exhaustive literature review of various recommendation systems, showcasing their methodologies, features, datasets, advantages, and disadvantages. The table contains distinct rows, each representing a different recommendation system approach, while the columns provide critical details about each approach.

The first column, **Literature**, briefly describes the title or name of the discussed recommendation system. The **Objective** column outlines the primary goal or purpose of the system. **Methodology** explains the techniques or algorithms employed in the system's implementation. **Features** refer to the data types utilized for making recommendations. **Datasets** specify the data sources used for training or testing the system.

Moreover, the table includes columns for **Advantages** and **Disadvantages**, highlighting the strengths and weaknesses of each approach. These insights provide valuable information for researchers, practitioners, and stakeholders interested in understanding recommendation systems' landscape, functionalities, and potential limitations.

Additionally, the table employs colour-coded rows to visually differentiate between different recommendation systems. For instance, green rows represent approaches focused on book recommendation systems, while red and blue rows denote different methodologies or emphases within broader recommendation system research. Yellow rows signify hybrid approaches combining multiple techniques.

By synthesizing a diverse array of recommendation system literature, this table serves as a valuable resource for understanding the current state of research in the field and the various strategies employed to address recommendation challenges across different domains and contexts.

TABLE 1
Literature Review

Literature	Objective	Methodology	Features	Datasets	Advantages	Disadvantages
A hybrid recommender system using artificial neural networks [12]	Predict movie ratings by combining collaborative filtering and content-based data	Artificial neural network classifier	User demographics, item content features, ratings	MovieLens	Combines multiple data sources, handles sparse data	Computationally expensive to train, prone to overfitting
Collaborative Deep Learning for Recommender Systems [13]	Predict item ratings by integrating deep learning on content with collaborative filtering on ratings	Stacked denoising autoencoder + matrix factorization	Item text content, user-item ratings	CiteULike, Netflix	Learns item representations from content, captures user-item relationships	Complex model, difficult to optimize
LFDNN: A Novel Hybrid Recommendation Model Based on DeepFM and LightGBM [14]	Predict clickthrough rates by combining DeepFM and LightGBM	Shallow FM model, deep neural network, gradient boosting trees	User-item interactions	Avazu, Criteo	Learns high-order feature crosses, handles sparse features	Complex Hybrid Architecture Deep Learning Recommendation Model for Personalization and Recommendation Systems
Wide & Deep Learning for Recommender Systems [15]	Combine memorization and generalization abilities for recommender systems	Joint training of wide linear models and deep neural networks	Sparse features, dense embeddings	Google Play app store data	Improves topical relevance and item diversity, efficient model serving	More complex architecture
Deep Neural Networks for YouTube Recommendations [16]	Personalized video recommendations on YouTube	Deep candidate generation model and deep ranking model	Video IDs, watch history, search queries, etc.	YouTube video data	Handles cold start, improves diversity and freshness	Computationally expensive, requires huge amounts of data
A Novel Approach for Book Recommendation Systems [17]	Generate association rules among books for recommendations	FPIntersect algorithm	Books purchased by users	Synthetic dataset	Reduces the number of database scans compared to Apriori	Specific to book recommendation
Improving a Hybrid Literary Book Recommendation System through Author Ranking [18]	Improve book recommendations by incorporating author preferences	Weighted hybrid of book and author collaborative filtering	User book ratings, authorship	LitRec	Incorporates author preferences, customizable weighting	Domain-specific to literary books

Book Recommendation System Using Opinion Mining Technique [19]	Recommend top-ranked books using opinion mining	Rating and polarity calculation	Book features, user reviews and ratings	Amazon book data	Incorporates review sentiment analysis	Focused on ranking rather than personalized recs
Book Recommendation System [20]	Book recommendation using association rules	ECLAT algorithm for frequent itemset mining	Books purchased together	Synthetic transaction data	Single database scan, computationally efficient	Rules may not capture personalized interests
Book Recommendation System through content-based and collaborative filtering method [21]	Hybrid book recommendation	Content-based, collaborative, keyword-based filtering	Book content, ratings	Custom website data	Incorporates multiple signals - content, ratings, keywords	Requires building a new book website
Deep Learning Recommendation Model for Personalization and Recommendation Systems [22]	Recommend items by combining deep learning on content with collaborative filtering	Deep candidate generation + deep ranking model	User features, item content features	Custom e-commerce data	Captures high-order feature crosses, custom architecture	Requires large amounts of data
Improving Content-based and Hybrid Music Recommendation using Deep Learning [23]	Improve content-based recommendations	Unified deep learning model	Learned from audio	Proprietary music dataset	Handles cold start, does not need CF	Computationally expensive
Deep content-based music recommendation [24]	Address the cold start problem	CNNs to Predict Latent Factors	Learned from audio	Million Song Dataset	Reasonable recommendation, handles cold start	Cannot predict song popularity
Hybrid recommendation system combined content-based filtering [25]	Improve the accuracy of recommendations	Combination of CF, content-based, SOM CF	Text and genres	MovieLens 100K	Improves accuracy, handles cold start	Computationally expensive, slower

Legend:

Deep Learning
Deep Neural Networks
Recommendation Algorithms
Content-Based Filtering

The article by Paradarami et al. (2017), highlighted in red, offers a comprehensive analysis of hybrid recommender systems that integrate deep learning, mainly through the use of Artificial Neural Networks (ANNs), to enhance the accuracy and effectiveness of recommendations in various contexts, including businesses [12]. This approach addresses significant challenges inherent in traditional recommender systems, such as the sparsity of data and the cold start problem, which often degrade the user experience due to inadequate personalisation.

The necessity of employing deep learning in the context of "LumiNovels," a hypothetical hybrid system for light novel recommendations, can be strongly justified based on the findings from Paradarami et al. Deep learning models, especially those utilising ANNs, can capture complex nonlinear relationships typical in user-item interaction data. These models can significantly outperform traditional methods by effectively integrating diverse data sources, including user reviews and metadata, to provide nuanced recommendations not solely dependent on historical ratings [12].

In addition, ANNs' flexibility allows for incorporating a wide range of features, from textual content in reviews to user demographic information, which could be particularly beneficial for recommending niche categories like light novels, where textual analysis of plot and genre preferences could play a crucial role. Thus, integrating deep learning into LumiNovels is beneficial and necessary to harness the full potential of the available data, improve recommendation quality, and enhance user satisfaction in a highly specialised market [12].

The paper by Covington, Adams, and Sargin (2016), also in red, explores the transformative impact of deep learning on YouTube's recommendation system, illustrating how it manages challenges associated with scale, freshness, and noise [16]. Deep learning has significantly enhanced the system's ability to handle YouTube's extensive user base and the dynamic nature of its video corpus, which continuously grows with new content every second.

In the context of "LumiNovels," a specialised platform for recommending light novels, the necessity of deep learning is evident from the improvements described in the YouTube system. Deep learning enables a nuanced understanding of complex and dynamic user interactions, essential for personalised recommendations in a domain as specific and varied as light novels. Deep neural

networks allow for integrating diverse data inputs—ranging from user preferences to contextual understanding of content—thus enabling more accurate predictions and personalised user experiences.

Moreover, deep learning's ability to manage and learn from large-scale data can be particularly advantageous for "LumiNovels." This approach would cater to the broad tastes and preferences within the light novel genre and adapt quickly to new trends and user interactions, thereby maintaining relevance and engagement in a rapidly evolving market [16].

The paper by Wang, Wang, and Yeung (2015), which is in purple, addresses the integration of collaborative filtering (CF) and deep learning within recommender systems through a novel approach named Collaborative Deep Learning (CDL). This model tackles the critical limitation of traditional CF methods, which suffer from sparsity issues in the user-item interaction data by leveraging deep representation learning. Through its hierarchical Bayesian framework, the CDL model enhances feature learning from content information and effectively intertwines it with CF for rating predictions, enabling it to surpass the limitations of previous models like matrix factorisation and shallow learning approaches.

For "LumiNovels," which focuses on recommending light novels, employing CDL can be particularly advantageous. The deep learning component of CDL is adept at extracting sophisticated features from complex, sparse datasets—standard in textual data like novels—while its collaborative filtering aspect captures and utilises the implicit relationships and preferences among users and items. This dual capability allows for more personalised and accurate recommendations, addressing new and existing items in the system, thus overcoming the cold start problem often faced by new entries in the dataset.

Additionally, integrating CDL into a system like "LumiNovels" allows for continuous adaptation and improved recommendation quality as more user interaction data becomes available. This adaptability is crucial for maintaining user engagement and satisfaction in dynamically changing content domains such as light novels[13].

The paper by Wang and Wang (2014) on "Improving Content-based and Hybrid Music Recommendation using Deep Learning", purple in the review table, provides a robust analysis of how deep learning techniques can significantly enhance content-based and hybrid music recommendation systems. The authors propose a model that unifies the feature extraction and recommendation process into a seamless operation, which overcomes the traditional limitations of content-based systems, such as the inability to handle new songs without prior usage history.

For "LumiNovels," a platform focused on light novel recommendations, utilising similar deep

learning techniques could be transformative. Deep learning can effectively manage the complexities associated with light novels' diverse and nuanced content, such as thematic elements, character development, and narrative style, which are crucial for making relevant recommendations. By applying a model similar to Wang and Wang's, "LumiNovels" could achieve superior performance in personalising recommendations and handling new titles effectively, thus enhancing user engagement and satisfaction by providing highly relevant novel suggestions based on deep, content-driven insights[23].

The paper, highlighted in green by Devika, Jisha, and Sajeev (2016), presents an innovative approach to a book recommendation system. This approach leverages a novel pattern mining algorithm known as the Frequent Pattern Intersect (FPIntersect) algorithm. This method addresses the limitations of traditional algorithms like Apriori by reducing the number of database scans required to generate associations among products, thus enhancing system performance and efficiency [17].

Applying deep learning could be pivotal for a specialised recommendation system like "LumiNovels," which focuses on light novels. Deep learning could enhance the FPIntersect algorithm by enabling a more nuanced analysis of user interactions and feedback. By incorporating deep learning, "LumiNovels" could effectively analyse complex user data, extract meaningful patterns from large datasets, and offer more accurate recommendations based on deep insights into user preferences and behaviours. Furthermore, deep learning can automate and optimise user feedback extraction and learning processes, improving the system's adaptability and responsiveness to new user data. Therefore, integrating deep learning into "LumiNovels" is beneficial and necessary for handling the sophisticated demands of modern recommendation systems in a content-rich domain like light novels [17].

Presented as green, The paper by Sohail, Siddiqui, and Ali (2013) presents an opinion-mining technique for book recommendations, utilising customer reviews to rank books in various computer science disciplines. This method relies on extracting features from customer reviews and applying weights to these features to calculate the rankings of books[19].

In the context of "LumiNovels," a system that recommends light novels, integrating deep learning could significantly enhance this opinion-mining approach. Deep learning could improve the accuracy of feature extraction from textual reviews, which is pivotal given the diverse and nuanced content of light novels that often includes complex themes and character arcs. Moreover, deep learning algorithms can handle the ambiguity and subtlety in language more effectively than traditional data mining techniques, enabling a more nuanced understanding of user sentiments and preferences.

Furthermore, deep learning could automate and refine the weighting and ranking processes by

learning from extensive user interactions and feedback datasets, potentially increasing the system's adaptability and accuracy. Therefore, incorporating deep learning into "LumiNovels" would enhance the efficiency of the opinion mining technique and ensure more personalised and contextually appropriate recommendations for users, which is crucial in the highly specialised and varied market of light novels [19].

The paper, in yellow, by Mathew, Kuriakose, and Hegde (2015), investigates the hybrid approach of combining content-based filtering (CBF), collaborative filtering (CF), and association rule mining to enhance book recommendation systems. The integrated method mitigates the common pitfalls of irrelevance and user dissatisfaction associated with existing systems and ensures more accurate and user-tailored recommendations [21].

Applying deep learning could be instrumental for LumiNovels, a platform that recommends light novels. Deep learning can effectively process and analyse the large datasets characteristic of user-item interactions in recommendation systems, improving the accuracy of traditional CF and CBF predictions. Furthermore, deep learning algorithms excel in feature extraction from complex data types like text, which is paramount when analysing light novels' intricate plots and themes.

Combining deep learning with the hybrid approach discussed by Mathew et al. could lead to significant improvements in "LumiNovels." By enabling more precise modelling of user preferences and item characteristics, deep learning can enhance the system's capability to recommend novels that resonate more deeply with individual users, thereby increasing user engagement and satisfaction[21].

Another paper, marked in yellow, by Afoudi et al. (2021) explores a hybrid recommendation system that combines content-based filtering, collaborative prediction, and an artificial neural network technique known as the Self-Organizing Map (SOM)[25]. This approach intends to use the strengths of multiple recommendation strategies to improve accuracy and efficiency, overcoming the limitations of individual methods when used in isolation.

In the context of "LumiNovels," which aims to recommend light novels, such a hybrid approach supported by deep learning is particularly essential. Deep learning facilitates the extraction of complex patterns and features from diverse data sources, including textual content and user interaction data, which are critical for understanding nuanced user preferences in literary genres. SOM in this hybrid system helps cluster similar items effectively, making the system more responsive and precise in its suggestions.

Moreover, the integration of deep learning allows "LumiNovels" to dynamically adapt to new content and changing user preferences, ensuring that the recommendations remain relevant and personalised. This adaptability is crucial in the light novel domain, where new works are frequently

published, and reader interests may evolve rapidly. Therefore, deep learning in "LumiNovels" enhances the recommendation quality and significantly contributes to user engagement and satisfaction by delivering tailored and contextually relevant novel suggestions [25].

2.2. Neural Network Classification

Neural network classification, a pivotal aspect of machine learning, is a captivating approach that categorizes input data into distinct classes. Its foundation lies in the intricate architecture of artificial neural networks (ANNs), which emulate the human brain's operations to decipher complex patterns within data.

An ANN comprises interconnected nodes organized into layers, each contributing to transforming input signals into meaningful output predictions. The hidden layers are central to the classification process, where the network learns to extract essential features from the input data through iterative adjustments of internal parameters, such as weights and biases [26]. This iterative refinement, facilitated by the backpropagation algorithm, allows the network to minimize the disparity between predicted and actual class labels, gradually improving its classification accuracy.

The versatility of neural network classification is truly inspiring. By harnessing deep architectures with multiple layers, neural networks can extract hierarchical representations of input features [27]. This eliminates the need for manual feature engineering and opens up new avenues for understanding complex datasets. Their scalability also equips them to handle vast volumes of high-dimensional data, a key requirement in today's big data applications.

However, alongside these strengths, neural network classification presents notable challenges. The inherent complexity of neural network models, characterized by numerous hyperparameters and architectural choices, poses a formidable obstacle to practical training and optimization [28]. The computational demands of training deep architectures further compound this challenge, necessitating access to substantial computational resources and time. Moreover, neural networks are susceptible to overfitting, particularly in scenarios with limited training data, where the model may memorize noise rather than discerning genuine patterns, leading to diminished generalization performance.

Furthermore, the black-box nature of deep neural networks [29], wherein the model's inner workings remain opaque, raises concerns regarding interpretability [27]. Understanding how neural networks arrive at their predictions, especially in intricate architectures with multiple layers, presents a significant obstacle to trust and adoption in critical domains where interpretability is paramount.

While neural network classification is a potent tool for solving complex classification tasks, its efficacy is tempered by complexity, computational resources, and interpretability considerations.

Addressing these challenges requires a balanced approach, incorporating advancements in model architecture, optimization techniques, and interpretability frameworks to unlock the full potential of neural network classification in real-world applications.

2.3. Content-Based Filtering

Content-based filtering stands out by offering a personalized approach to suggesting items. It does this by considering both the intrinsic characteristics of the items and the user's preferences. This is in contrast to collaborative filtering, which relies solely on historical user interactions [3]. By using the inherent features of items, content-based filtering can make recommendations that are uniquely tailored to individual tastes.

Content-based filtering is a practical tool that involves extracting and analyzing item attributes, such as textual descriptions, metadata, or image features. These attributes are used to build profiles of user preferences, which are then matched against the characteristics of available items. This process allows content-based filtering to recommend items that closely align with the user's interests, providing a personalized and targeted recommendation experience.

One critical advantage of content-based filtering is its ability to address the cold start problem. Traditional collaborative filtering approaches need help making recommendations for new or niche items with limited user interactions. By relying solely on item attributes, content-based filtering circumvents the need for historical user data, making it particularly well-suited for scenarios with sparse or incomplete user-item interaction matrices [30].

Furthermore, content-based filtering offers transparency and interpretability, as recommendations are grounded in explicit item features that users can readily comprehend. This transparency fosters user trust and engagement with the recommendation system, enhancing user experience and satisfaction.

While content-based filtering has its advantages, it also has limitations. One significant drawback is the potential for overspecialization, where the system may fail to introduce users to novel or unexpected items. This can create filter bubbles, limiting serendipitous discovery and diversity in recommendations. However, these challenges can be managed with careful system design and user feedback.

Moreover, content-based filtering is inherently constrained by the quality and availability of item attributes. In domains where item features are sparse, noisy, or poorly annotated, the effectiveness of content-based filtering may be compromised, hindering its ability to generate accurate and diverse recommendations.

Despite these challenges, content-based filtering remains valuable in the recommendation system

toolkit. It offers a principled approach to personalized recommendation that complements collaborative filtering methods. By leveraging item attributes to understand user preferences, content-based filtering enables recommendation systems to deliver relevant and engaging recommendations tailored to individual tastes, enriching the user experience and driving user satisfaction.

2.4. Hybridisation

The project aims to integrate neural network classification with content-based filtering to develop a hybrid recommendation system that addresses the limitations of both approaches while leveraging their respective strengths. By combining these techniques, the system seeks to enhance recommendations' accuracy, diversity, and personalisation.

Neural network classification offers powerful capabilities for capturing complex patterns within data. Still, it may need help with interpretability and the cold start problem, mainly when dealing with sparse or noisy datasets. Conversely, content-based filtering addresses the cold start problem and provides transparent recommendations based on item attributes. However, it may suffer from overspecialisation and limited diversity in recommendations.

The recommendation system aims to overcome these challenges by hybridizing these approaches. The neural network classification component will classify the entries based on their characteristics and user preferences, providing a foundation for personalised recommendations. Meanwhile, the content-based filtering component will enhance the recommendation process by leveraging the classified data to identify the top 5 matching results based on user input.

This hybrid approach offers several advantages. First, it leverages the predictive power of neural networks to classify entries accurately, improving the quality of recommendations. Second, by incorporating content-based filtering, the system can address the cold start problem and provide transparent, interpretable recommendations based on item attributes. Third, the hybrid approach enhances the diversity of recommendations by combining the strengths of both techniques, thereby enriching the user experience and fostering serendipitous discovery.

The project's focus on hybridizing neural network classification with content-based filtering represents a promising direction in recommendation system research. It offers a principled approach to personalised recommendation that combines the predictive capabilities of neural networks with the transparency and diversity of content-based filtering.

2.5. Requirements

For the project, Java has been selected as the primary programming language for its robustness, scalability, and versatility in developing complex software systems. While Python is often favoured

for its ease of use and extensive library support, particularly in neural networks, Java offers several distinct advantages that make it the preferred choice for this project [31].

Java's strong typing and object-oriented nature provide a solid foundation for building large-scale software applications, ensuring code reliability and maintainability. Its platform independence allows the project to run seamlessly across different operating systems, facilitating deployment and compatibility.

Moreover, Java's performance and efficiency, particularly in handling multithreading and memory management, make it well-suited for computationally intensive tasks, such as training neural networks and processing large datasets. While Python may offer superior libraries for neural network development, Java's performance optimizations and execution speed outweigh these advantages, especially in scenarios where real-time processing and responsiveness are critical.

For neural network development, the project will leverage deeplearning4j, a powerful deep-learning library for Java that provides comprehensive support for building and training neural network models. This library offers a wide range of functionalities, including various neural network architectures, optimization algorithms, and parallel processing capabilities, making it an ideal choice for implementing the neural network component of the recommendation system.

In addition to deeplearning4j, the project will utilize openCSV, a lightweight and efficient library for reading and writing CSV files. This library simplifies data handling tasks, enabling seamless integration of external datasets into the recommendation system. Furthermore, Apache Commons math will be employed for content-based filtering, offering essential mathematical functions and algorithms for processing and analyzing item attributes.

Lastly, GitHub will be the project's version control system, facilitating collaborative development, code sharing, and version tracking. With GitHub, project contributors can efficiently manage changes, track issues, and ensure code quality through code reviews and continuous integration practices.

Java's robustness, performance, and extensive library support make it the ideal choice for developing the recommendation system. Leveraging deeplearning4j, openCSV, and Apache Commons math, along with GitHub for version control, the project aims to deliver a reliable, scalable, and efficient solution for a personalized recommendation.

3. Methodology

3.1. System Diagram

Fig.1 provides an insightful overview of the project's system architecture, delineating the sequential flow of operations. Initially, data retrieval occurs from the MyAnimeList API, where information regarding light novels is accessible. Subsequently, the Java code processes the JSON responses obtained from the API, undertaking crucial data-cleaning tasks before storing the entries into a CSV file for further analysis. This dataset undergoes a multifaceted examination process aimed at extracting meaningful insights. The data is segmented into distinct categories: ID, genres, synopsis, and authors' names. However, handling textual data such as names necessitates the application of Natural Language Processing (NLP) techniques to transform it into a numerical format suitable for neural network input [32]. Following data preprocessing, the model embarks on a comprehensive training, evaluation and optimization cycle, iteratively refining its parameters to enhance performance and visualize evaluation metrics effectively. Upon completing this cycle, the user interface prompts users to input a name, genre, or synopsis, which undergoes preprocessing before being fed into the neural network for relabeling. Subsequently, the network assigns labels to the input data, paving the way for content-based filtering techniques to recommend the top 5 most similar entries based on the user's input, thereby facilitating personalized recommendations.

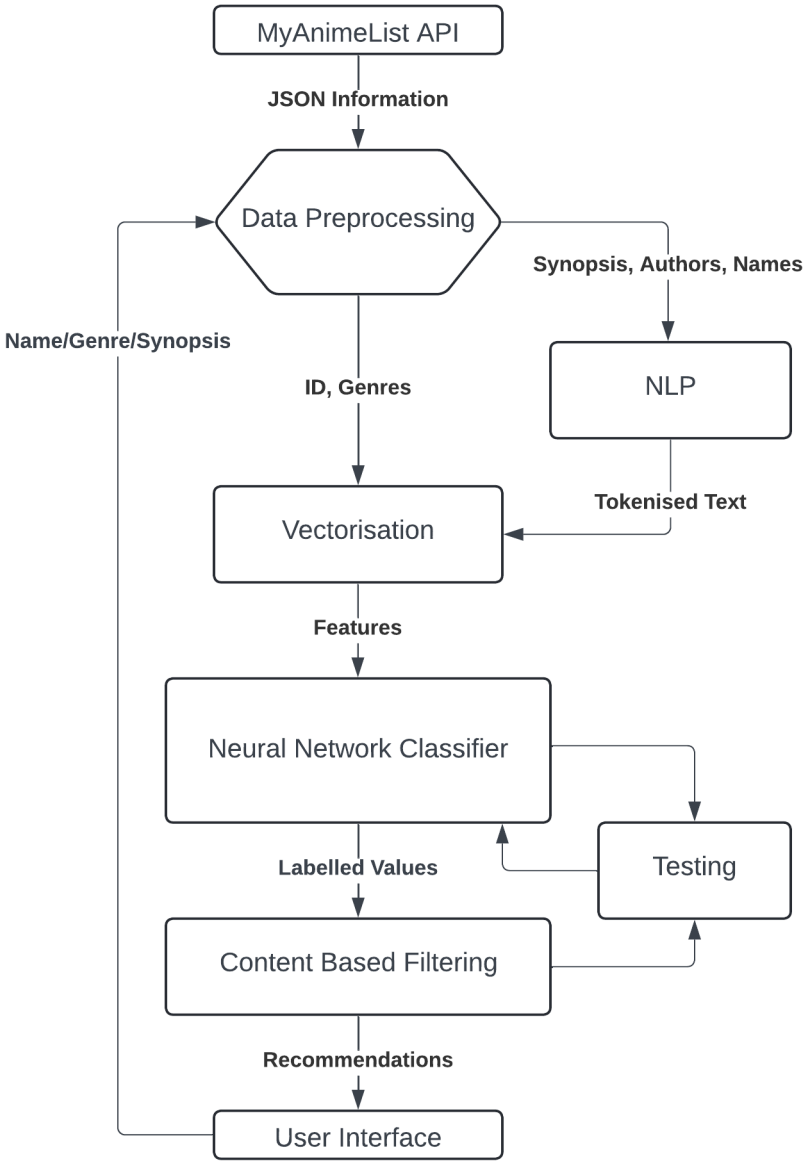


Fig 1: LumiNovels System Diagram.

3.2. MyAnimeList API Dataset

The MyAnimeList API [33] dataset emerges as a pivotal resource within this academic endeavour, particularly concerning the analysis of light novels where comprehensive datasets are scarce; despite exhaustive searches across platforms such as Kaggle and HuggingFace, existing options failed to meet the stringent criteria for dataset robustness and completeness. In this context, the MyAnimeList API offers a vast repository of information encompassing not just anime and manga but also light novels.

One of the salient features of the MyAnimeList API lies in its accessibility. Unlike many other APIs, MyAnimeList provides free access for educational purposes, empowering researchers to retrieve detailed information about light novels with relative ease. This accessibility proves pivotal in constructing a recommendation system that harnesses diverse data sources to deliver accurate and personalized recommendations.

Implementing code to interface with the MyAnimeList API revolves around utilizing HTTP GET requests, a standard protocol for retrieving data from web servers. The URL for the API call is provided in Fig.2; notice the endpoint called is **/v2/manga/**, which is the only way to access the light novel data, following the well-maintained API documentation. However, accessing the dataset necessitates the generation of a public key on the MyAnimeList website, which acts as a credential for API access. Registration on the MyAnimeList website is a prerequisite for users to create a public key. Once authenticated, the API responds with a JSON-formatted payload containing a wealth of information about each light novel entry.

APIextractor.java

```
// API Endpoint URL for manga ranking
String apiUrl = "https://api.myanimelist.net/v2/manga/ranking?offset=" + Integer.valueOf(offset)
        + "&ranking_type=lightnovels&fields=id,title,mean,genres,authors,synopsis,alternative_titles&limit=500&nsfw=true";// limit=500
offset += 500; // the number of instances of data in csv file
```

Explain

Fig 2: API call.

To streamline the data retrieval process and mitigate potential errors, a strategic choice is made to prioritize the "ranking" field. This field enables fetching up to 500 entries at once, significantly reducing the risk of encountering rate-limiting restrictions such as Error 403 (Suspected DoS attack) when making multiple requests within a short timeframe. Opting for bulk retrieval over individual requests ensures efficiency and simplicity in data acquisition, thereby conserving time

and computational resources.

In selecting fields from the API response, comprehensive coverage of critical attributes essential for recommendation system development is prioritized. These attributes include the unique identifier (ID) for each entry, the main title of the light novel, the mean score provided by users on the MyAnimeList website, an array of genre IDs and names, author IDs, a synopsis of the plot, and alternative titles. Each attribute plays a crucial role in understanding the light novels' content, popularity, and thematic elements, laying the groundwork for accurate and relevant recommendation generation. A sample of the JSON response is provided in fig.3, displaying the previously discussed fields and their values. The response represents one entry or light novel, including alternate names, such as the Japanese version of the title.

With its accessibility, comprehensive coverage, and strategic data retrieval approach, the MyAnimeList API dataset exemplifies the commitment to excellence in recommendation system research, and the values are preprocessed before being stored as a dataset in a CSV file.


```
{"data":[{
  "node":{
    "id":14893,
    "title":"Monogatari Series: First Season",
    "main_picture":{
      "medium":"https:\/\/cdn.myanimelist.net\/images\/manga\/2\/279887.jpg",
      "large":"https:\/\/cdn.myanimelist.net\/images\/manga\/2\/279887l.jpg"},
    "mean":8.93,
    "genres":[
      {"id":1,"name":"Action"},{"id":4,"name":"Comedy"},
      {"id":7,"name":"Mystery"},{"id":22,"name":"Romance"},
      {"id":37,"name":"Supernatural"},{"id":32,"name":"Vampire"}],
    "authors":[
      {"node":{"id":5254},"role":"Story"},
      {"node":{"id":8594},"role":"Art"}],
    "synopsis":"This is a story, a \"ghostory\" of sorts, about scars that bond, monsters that haunt,
    and fakes that deceive.\n\nThe story of Koyomi Araragi begins through a fateful encounter with
    the all-powerful, blonde-haired, \"hot-blooded, iron-blooded, and cold-blooded\" vampire,
    later introduced as Shinobu Oshino. Their tragic rendezvous results in the end of Araragi's life
    as a human and his subsequent rebirth as a vampire\u2014a monster.
    However, this encounter is only the start of his meddlings with the supernatural.
    Koyomi's noble personality ultimately sees him getting further involved in the lives of others
    with supernatural afflictions. This is his desperate attempt at returning to a normal human life,
    in a paranormal world filled with nothing but tragedy, suffering, and inhumanity.
    \n\n[Written by MAL Rewrite]\n\nThis entry includes the first season of the Monogatari Series.",
    "alternative_titles":{
      "synonyms":["Bakemonogatari","Monster Tale","Kizumonogatari","Wound Tale",
        "Nisemonogatari","Fake Tale","Nekomonogatari: Kuro","Nekomonogatari: Black"],
      "en":"",
      "ja":"\u3008\u7269\u8a9e\u3009\u30b7\u30ea\u30fc\u30ba
      \u30d5\u30a1\u30fc\u30b9\u30c8\u30b7\u30fc\u30ba\u30f3"}},
      "ranking":{"rank":1}}
```


Fig 3: JSON response.

3.3. Data Preprocessing

In the data preprocessing stage, the code undergoes a meticulous process to prepare the dataset for subsequent analysis and model training. Initially, a loop iterates through all entries in the dataset, aiming to construct a comprehensive dictionary of all existing genres within the dataset. This initial step proves instrumental in facilitating subsequent operations. Following this, a second loop is executed wherein each entry is processed. A binary representation of genres is generated for each entry, where each genre is encoded as either 1 or 0. If a particular genre is present in the light novel, the corresponding value for that genre of the entry is set to 1; otherwise, it is set to 0. This binary encoding results in a sequence of 0s and 1s representing the presence or absence of each genre within the entry. Significantly, this approach mitigates potential issues, such as cases where an entry has no genres. In such scenarios, setting all genres as 0 prevents information loss and ensures consistency in data representation.

The entries are initially stored in a HashMap structure to facilitate efficient data management and retrieval. This data structure is chosen for its ability to retrieve and manipulate data swiftly. Each entry is stored as a key-value pair, with the entry's unique identifier (ID) as the key. Since IDs are inherently unique, they provide an efficient means of indexing and referencing entries within the dataset. The remaining attributes of each entry, including genre representations, are stored as values associated with their respective IDs. This organizational approach streamlines subsequent operations, mainly when writing the preprocessed data to a file, as the HashMap structure enables rapid retrieval and association of entry attributes during the file-writing process.

3.4. Dataset Analysis

The dataset analysis reveals a comprehensive breakdown of light novel entries and their corresponding genres, facilitated by the adept use of OpenCSV. This Java library stands out due to its robust parsing capabilities, which are essential for handling large and varied datasets. OpenCSV was selected over alternatives like Apache Commons CSV primarily because of its more straightforward, intuitive API and faster performance when dealing with extensive records. The dataset comprises 10,343 unique light novels encompassing a diverse range of 71 genres. These genres span traditional themes such as school, romance, and comedy, and extend into niche categories like mecha (giant robots), isekai (adventures in alternate worlds), and horror, not shying away from the inclusion of NSFW content like gore.

Further insights into the data are visually represented through several figures developed using Java technology. Fig.4, constructed on a JFrame applet, leverages the JFreeChart library to illustrate the popularity ratings of each genre. This choice was driven by JFreeChart's superior graphical

capabilities, which offer extensive customization options and precise, crisp rendering of charts. It is preferable to other charting libraries, such as Google Charts or Chart.js, which might require more complex integrations for Java-based applications. The figure highlights Avant Garde as the highest-rated genre with a score of 6.86, followed by Organized Crime at 5.92 and Mahou Shoujo (Magical Girls) at 5.68. Note that the scores are on a scale from 0.0 to 10.0.

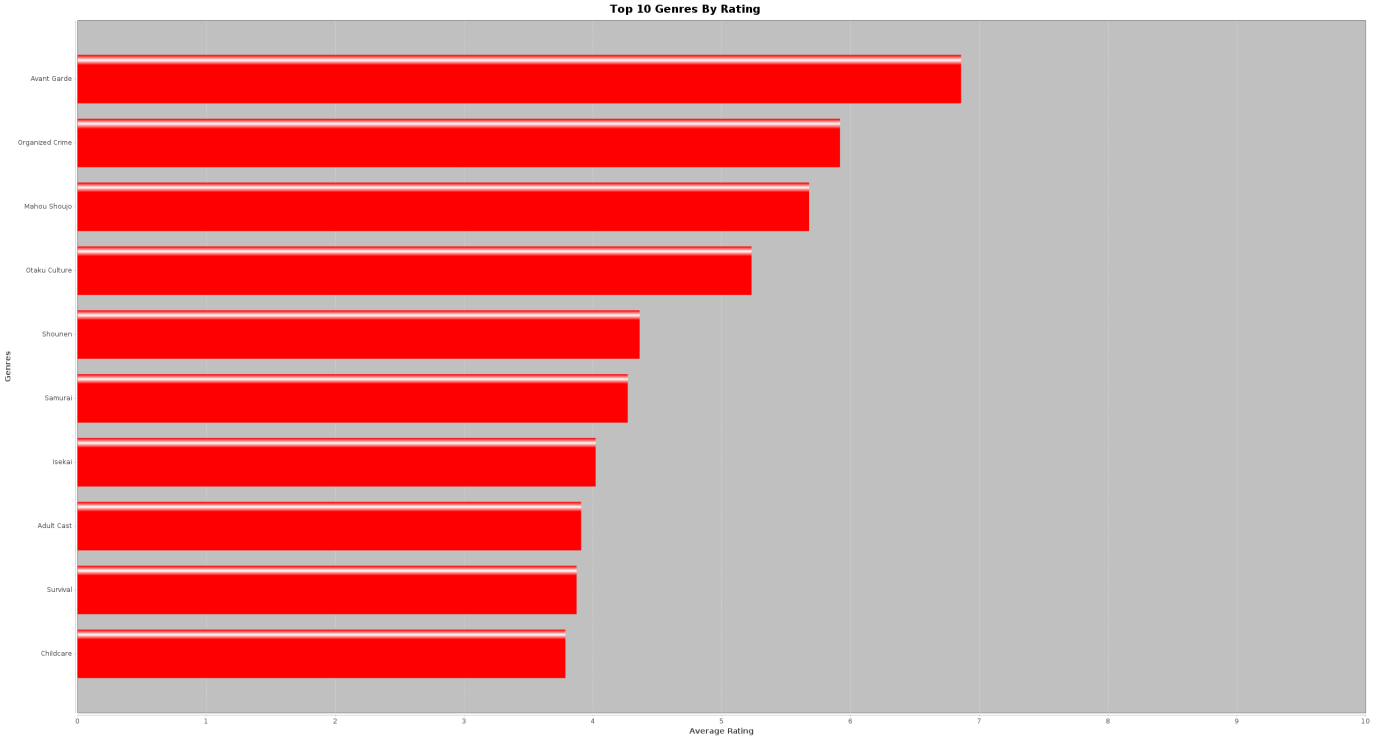


Fig 4: Top 10 genre ratings. Full diagram provided in appendix 10

Similarly, Fig.5 utilizes the same technologies to display the genre distribution across the novels. It reveals that Romance features in 34.91% of the novels, making it the most prevalent, closely followed by Fantasy at 33.45%, and Hentai, marked as an NSFW category, appearing in 27.44% of the light novels. This detailed portrayal not only elucidates the popularity of various genres but also underscores the breadth of thematic exploration within light novels. These visual representations effectively convey the data's dimensions, providing a clear snapshot of current trends and preferences within the light novel landscape.

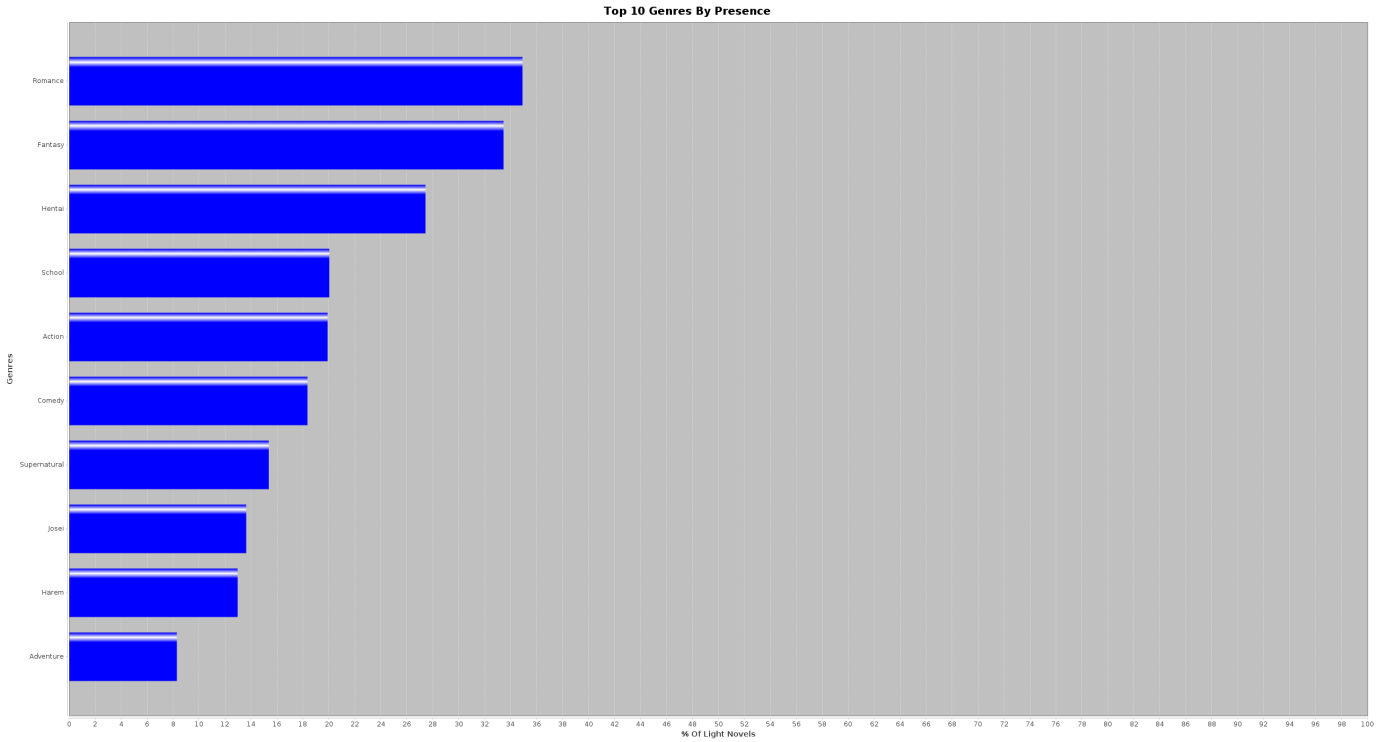
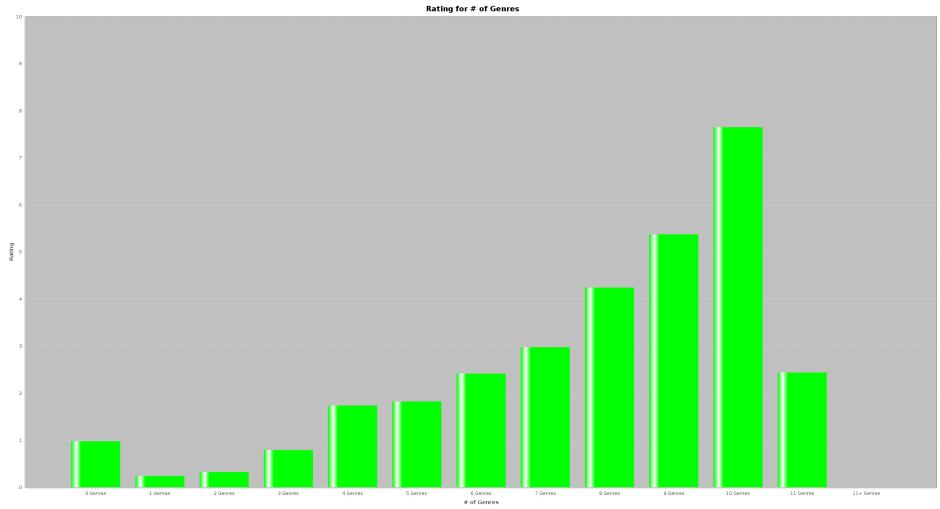


Fig 5: Top 10 most frequent genres, full diagram provided in appendix B.

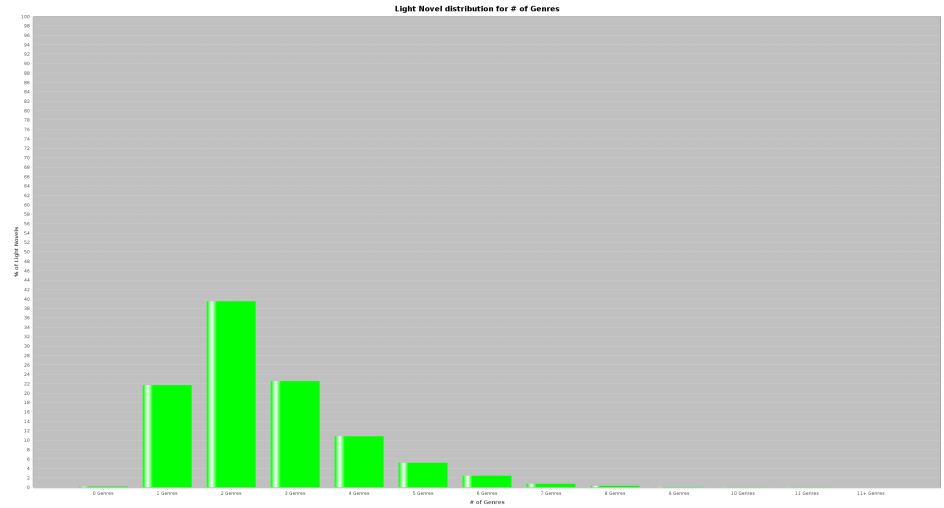
Fig.6a offers a detailed visualization of the average scores associated with the number of genres per light novel, unveiling a discernible trend: novels featuring a higher count of genres generally achieve better ratings. Specifically, light novels with a singular genre register a low average rating of 0.236, whereas those encompassing 10 genres reach a much higher average rating of 7.647. Despite this pattern, anomalies exist, such as light novels without any genre designation scoring 0.975. This score, although minimal, still surpasses the average ratings of light novels with up to three genres. Interestingly, the proportional increase in ratings plateaus with novels containing more than 10 genres; for instance, those with 11 genres drop to an average rating of 2.437, and novels with more genres see a near negligible average rating.

In contrast, Fig.6b illustrates the distribution of light novels across different genre counts, showing a reverse trend. While 39.49% of light novels incorporate two genres, the proportion diminishes as the number of genres increases, with only 0.03% of novels featuring 10 genres. Additionally, 0.15% of the novels are categorized under no specific genre, and a mere 0.03% have 11 or more genres.

These observations from the two graphs suggest a notable bias in the relationship between genre diversity and light novel ratings, indicating that a broader genre spectrum tends to correlate with higher scores up to a certain point, after which the benefit diminishes or even reverses.



(a) Average rating for the number of genres.



(b) Light novel distribution for the number of genres.

The bias identified in the distribution of genres and their associated ratings, as depicted in Figs.6a and 6b, aligns with insights garnered from previous research presented in the literature review. This research suggests that the conventional popularity metric, often gauged by average ratings, might not be the most reliable indicator for evaluating the quality or success of light novels.

The trend where novels with a higher diversity of genres generally receive better ratings highlights an intrinsic bias in rating systems; novels that cover more genres tend to appeal to a broader audience, potentially inflating their perceived quality or popularity. However, the anomaly observed—where novels with an excessive number of genres see a significant drop in ratings—further

complicates the use of popularity as a measure. This indicates that beyond a certain point, the addition of more genres does not necessarily enhance a novel's appeal or quality and might detract from it.

These findings suggest that relying solely on popularity metrics such as average ratings could lead to misleading conclusions about light novels' actual appeal or quality. This could affect stakeholders, including authors, publishers, and readers, who rely on these metrics for decision-making.

3.5. Natural Language Processing

Natural Language Processing (NLP) is a pivotal domain within artificial intelligence that facilitates the interaction between computers and humans using natural language. The essence of NLP lies in its ability to process and analyze large amounts of natural language data [34], making it invaluable across various applications, from sentiment analysis and chatbots to machine translation and speech recognition. Despite its numerous applications, NLP has drawbacks, including challenges like ambiguity in language, the need for extensive computational resources, and difficulties in handling idiomatic expressions that can affect the accuracy and efficiency of NLP systems.

A practical example of implementing NLP in Java is using the openNLP library, which provides tools for natural language text processing. This library includes several modules, such as the Tokenizer, POSTagger, and Lemmatizer, each serving a specific function in the text analysis pipeline.

Tokenizing is breaking down text into smaller components, or tokens, typically words or phrases. This step is foundational in NLP as it prepares raw text for further processing and analysis. **POSTagging**, or part-of-speech tagging, involves assigning word types, such as nouns, verbs, or adjectives, to each token. This categorization helps understand each word's role within a sentence, enhancing the processing accuracy for subsequent tasks such as parsing and sentiment analysis. **Lemmatizing** involves stripping words down to their base or dictionary form, a step that normalizes word variations and enhances the uniformity of input text for analysis.

For effective text preprocessing, refining the input data by converting the text into lemmatized tokens and removing stopwords is essential. Stopwords are words that, despite their frequency in the text, offer little to no value in terms of understanding or analysis. These often include commonly used prepositions, conjunctions, and pronouns. In the context of this analysis, specific parts of speech tags such as "CD" (cardinal numbers), "DT" (determiners), "VB" (verbs), "WP" (wh-pronouns), "RB" (adverbs), "RP" (particles), "WDT" (wh-determiners), "IN" (prepositions or subordinating conjunctions), "TO" (to), and "CC" (coordinating conjunctions) are identified for the removal of stopwords. Further, it is noticed that tokens falling under these tags are predominantly

romanized Japanese words or typical English stopwords such as "and", "in", and "is", as well as words like "can" and "source", which appear ubiquitously across texts, rendering them ineffective for numerical conversion and subsequent analysis.

Excluding Romanized words from analysis is practical in addressing the challenges of processing them since the openNLP bins are optimized primarily for English. This approach simplifies the processing while accommodating the limitations of the toolset, ensuring that the focus remains on effectively analyzing the content that can be accurately processed.

By leveraging the capabilities of the openNLP library in this structured manner, the text preprocessing for NLP tasks becomes more targeted and efficient, paving the way for more accurate and meaningful analysis in subsequent stages of the NLP pipeline.

3.6. Genre Merging

In the expansive domain of natural language processing applied to the analysis of light novels, substantial discrepancies in text length across entries are noted, with the most extended synopsis comprising 2785 words, while the shortest spans only 16. Moreover, a significant portion of entries, approximately 71.63% or 7409 synopses, are empty, highlighting a severe data sparsity issue. Additional textual elements such as titles, alternate titles, and author names are incorporated to augment the dataset, providing a broader textual base for analysis.

Upon deploying a neural network, certain classes and features initially overlooked become apparent, suggesting that a reduction in class count might mitigate data sparsity and enhance model performance. Given that the initial feature set was confined to genres, an innovative approach to condense these categories through merging could refine the accuracy of genre classification.

To address the challenge of genre amalgamation within the dataset, K-means++ clustering emerges as a strategic solution, offering a refined approach to the traditional K-means clustering algorithm. K-means, a popular clustering technique, partitions data into K distinct clusters based on feature similarity. However, its effectiveness is often hindered by the initial random selection of cluster centres, which can lead to variability in the results and slow convergence towards optimal clustering.

K-means++ clustering addresses these shortcomings by introducing a more competent initialisation phase that systematically selects initial cluster centres before proceeding with the standard iterative algorithm. The first step in K-means++ involves selecting a single cluster centre uniformly at random from the dataset, establishing a foundation that aims to be representative. Subsequent centres are chosen not randomly from the remaining data points but with careful consideration of their distance from the existing centres. Specifically, each subsequent centre is selected with a

probability proportional to the square of its distance from the nearest already chosen centre. This method drastically reduces the chance of selecting outliers as initial centres. It ensures a more spread-out selection, which helps minimise the sum of squared distances between data points and their nearest cluster centre.

This improved initialisation process enhances the overall efficiency of the clustering operation, leading to faster convergence and more consistent results across different runs of the algorithm. Clusters formed through K-means++ are generally more balanced and reflect an actual partitioning of the dataset based on underlying patterns.

However, despite these advantages, K-means++ retains certain limitations inherent to K-means, particularly its sensitivity to noise and outliers. Since cluster centres are initially dependent on actual data points, any noise or outlier can disproportionately influence the shape and spread of the resulting clusters. This sensitivity can sometimes result in skewed distributions, especially in datasets where outliers are not distinguishable from core data points, potentially impacting the reliability of the clustering outcome. Thus, while K-means++ significantly improves the robustness and effectiveness of cluster analysis, careful preprocessing of data to mitigate noise and identify outliers remains crucial for achieving optimal results.

3.7. Text Vectorisation

Vectorization is a critical preprocessing step in natural language processing, particularly when preparing data for neural networks and other machine learning models, which require input in numerical form, among the various methods for text vectorization, word count and TF-IDF (Term Frequency-Inverse Document Frequency) are commonly used approaches.

Word Count Vectorization tallies the number of times each word appears in a document. This approach is straightforward and effective in cases where the sheer frequency of words indicates their relevance. However, it also has significant drawbacks; primarily, it tends to emphasize common words that may not contribute meaningful information regarding the document's content. This can lead to a skewed representation of text data, where frequent but unimportant words dominate.

TF-IDF Vectorization addresses some of the shortcomings of word count vectorization. TF-IDF considers the frequency of words in a document (TF) and how rare the word is across all documents in the corpus (IDF) [35]. This reduces the weight of words that occur frequently across documents, deeming them less informative than words that occur frequently in a few documents. The equation for calculating the TF-IDF score for a word in a document is given by Eq.1:

$$w_{i,j} = tf_{i,j} * log(\frac{N}{df_i}) \quad (1)$$

where: $tf_{i,j}$ is the term frequency of term i in document j, N is the total number of documents, and df_i is the document frequency, so the number of documents containing i.

This method is preferable because it balances the frequency of terms with their relevance, giving higher scores to terms distinct from a document. However, TF-IDF vectorization also increases dimensionality and sparseness, as it generates many zeros, particularly for documents with few words, which can be problematic for certain models. High dimensionality can be mitigated using dimensionality reduction techniques such as Principal Component Analysis (PCA) [36], which simplifies the data while retaining important information.

During the vectorization process, the system meticulously traverses the dataset to establish two essential dictionaries: housing all unique words and mapping each word to its corresponding document frequency. This meticulous approach enables the calculation of TF-IDF scores for individual words, facilitating their vectorization within each entry. Notably, this vectorization operates at the word level, eschewing n-gram vectorization to avoid exacerbating computational complexity. Despite the potential augmentation of computational demands, this word-level vectorization method substantially enhances the interpretability and efficacy of the resulting vector representations. Discerning each document's unique contents more nuanced distinctions between entries based on their textual characteristics. Notably, while genres are already binary vectorized, represented by 0s and 1s, the remaining textual data undergoes vectorization. This includes the synopsis, the title of the light novel, its alternate titles, and the list of authors, ensuring a comprehensive representation of textual features in the vector space.

4. Development

4.1. Neural Network Configuration

The configuration of a neural network in a multi-class classification scenario involves selecting a series of parameters and methods that align with the specific needs and characteristics of the data. For the development environment, the Deeplearning4j Java library is chosen, catering specifically to Java applications and providing a robust platform for constructing and deploying neural networks.

Stochastic Gradient Descent (SGD) is employed as the updater algorithm for this neural network. SGD is a popular optimization method in machine learning that iteratively updates the model parameters, typically weights, to minimize the error between the predicted and actual outcomes. Unlike standard gradient descent, which computes the gradient using the entire dataset—thus being computationally expensive and slow—SGD improves efficiency by approximating the gradient using a randomly selected subset of data. This stochastic nature significantly speeds up the convergence, especially with large datasets, making it ideal for deep learning. However, the trade-off comes in the form of potentially higher variance in the update path, which can make the convergence path toward the minimum less stable and sometimes result in suboptimal solutions.

To optimize model performance, the Xavier initialization method is used to set the initial weights. This technique, named after Xavier Glorot, is designed to maintain the variance of activations and back-propagated gradients throughout the network. By keeping the scale of gradients roughly the same in all layers, Xavier initialization helps in preventing the vanishing or exploding gradients problem, common in deep networks. This is particularly critical in deep learning configurations to ensure that each layer learns at a similar pace during training.

Given the specific range of the input values from 0 to 1, certain activation functions are more suitable. The ReLU (Rectified Linear Unit) activation function is selected for the hidden layers. ReLU is defined as:

$$f(x) = \max(0, x) \tag{2}$$

, providing a linear response for positive inputs and zero for negative inputs. Its simplicity allows for faster computation and alleviates the vanishing gradient problem, which is frequent in networks using sigmoid or tanh functions. For the output layer, the Sigmoid activation function is utilized, which maps the output values between 0 and 1, making it an appropriate choice for binary classification tasks within each class. The sigmoid function is defined as

$$f(x) = \frac{1}{1 + e^{-x}} \tag{3}$$

, smoothly varying and non-linear, ideal for producing probabilities as output.

For the loss function, which guides the optimization of the network parameters, Binary Cross-Entropy (XENT) is employed. This function is specifically suited for outputs that represent probabilities between 0 and 1, typically used in conjunction with the sigmoid activation function. Binary Cross-Entropy measures the distance between the network's predicted probabilities and the actual binary labels (0 or 1), effectively penalizing predictions that diverge from the actual labels. The function is mathematically represented as

$$-[y \log(p) + (1 - y) \log(1 - p)] \tag{4}$$

, where y is the binary true label, and p is the predicted probability. It is designed to create a strong penalty for a prediction that is wrong and confident, thus focusing the training on accurate and confident predictions.

These initial configurations of the neural network, leveraging Deeplearning4j and specific choices in optimization and architecture, are aligned to effectively handle the challenges of multi-class classification within the computational constraints of the Java environment. A scaled down diagram is provided to show the structure of the network in Fig.7.

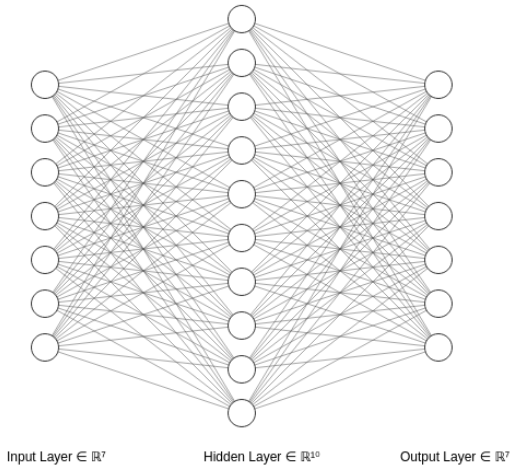


Fig 7: Scaled Neural Network Diagram.
Input Layer
Hidden Layer - ReLU
Output Layer - Sigmoid

4.2. Content Based Filtering Approach: Cosine Similarity

A content-based filtering approach enhances user experience in personalised recommendation systems by suggesting items closely aligning with their interests. This technique leverages a hybrid system that integrates a neural network with content-based filtering. The neural network is crucial for processing and classifying entries based on user information through a command line interface, be it an ID, name, synopsis, or genre. After preprocessing, this data is classified, and the content-based filter then uses the resulting labels to identify and recommend the top five entries.

Cosine similarity is employed to determine the similarity between the user's interest and the entries in the database [37]. Cosine similarity measures the cosine of the angle between two non-zero vectors of an inner product space, given by the equation:

$$\text{Cosine Similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} \tag{5}$$

In this equation, $\mathbf{A} \cdot \mathbf{B}$ is the dot product of vectors $\mathbf{A}$ and $\mathbf{B}$, and $\|\mathbf{A}\|$ and $\|\mathbf{B}\|$ are the magnitudes (or Euclidean norms) of the vectors. This similarity measure is particularly effective in handling variables of different magnitudes, focusing solely on the angle between vectors, making it robust against varying document lengths and frequency of terms.

Cosine similarity's key advantage over measures like linear similarity or Euclidean distance is its focus on vector orientation rather than magnitude, which is crucial for text data where document length can vary significantly and does not necessarily impact semantic similarity [38]. However, cosine similarity does have its challenges, particularly with sparse data. In such scenarios, where vectors are high-dimensional yet sparse, cosine similarity might overemphasise the angle between the vectors rather than their overlapping terms. This can lead to recommendations that are not as contextually relevant as possible if the vector space were denser. Additionally, cosine similarity assumes all dimensions are equally important, which may only sometimes be accurate and could skew recommendations.

5. Results and Evaluation

5.1. Initial Evaluation Metrics

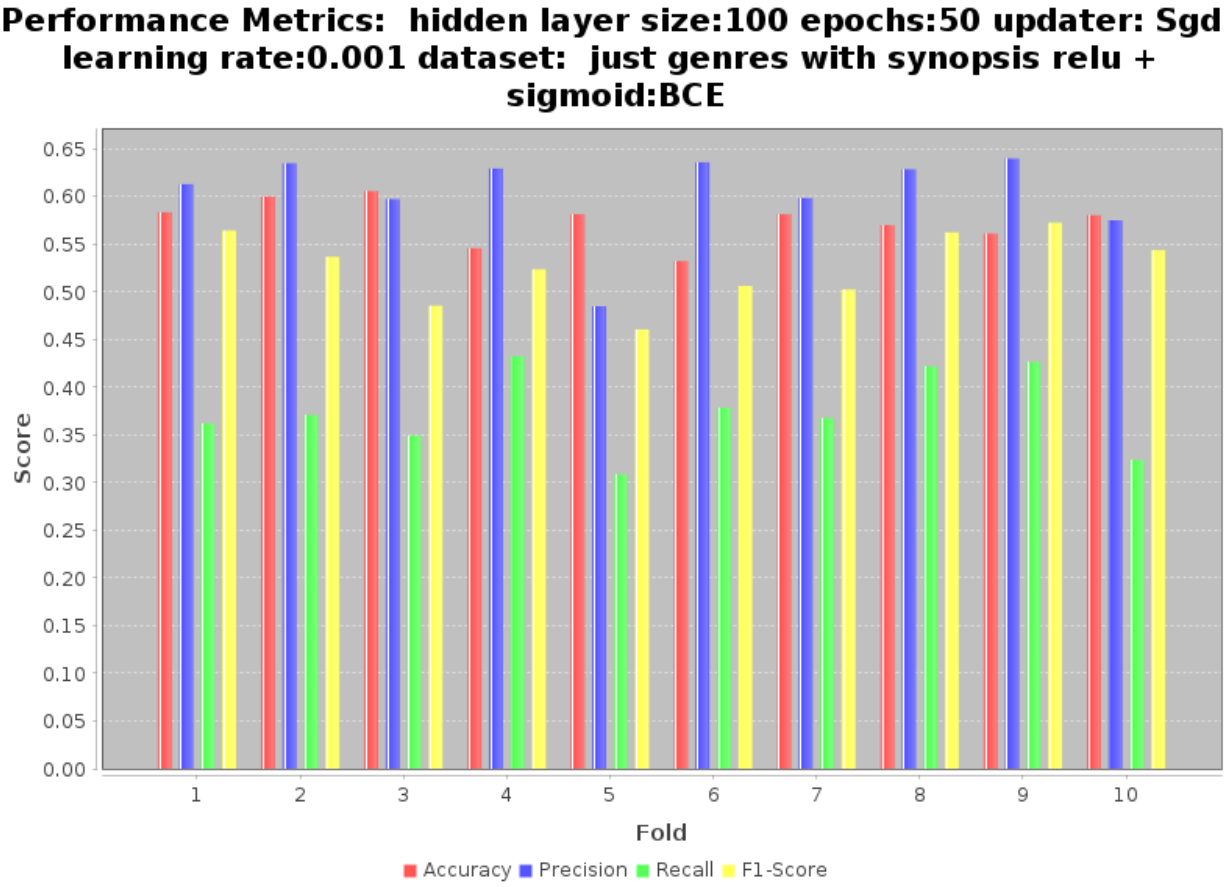


Fig 8: Evaluation metrics of neural network with SGD.

Several key metrics are employed to gauge recommendation systems' performance and effectiveness. These metrics, including Accuracy, Precision, Recall, and F1-score, offer insights into different aspects of the system's performance, allowing for a comprehensive evaluation of its capabilities.

Accuracy measures the proportion of correct predictions among all predictions made by the system [39]. Mathematically, it is calculated as the ratio of the number of correct predictions to the total number of predictions:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{True Positives} + \text{False Positives} + \text{True Negatives} + \text{False Negatives}} \tag{6}$$

Precision represents the proportion of true positive predictions among all positive predictions made by the system [40]. It is calculated as:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \tag{7}$$

Recall (also known as Sensitivity) measures the proportion of true positive predictions among all actual positive instances in the dataset [41]. Mathematically, it is expressed as:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \tag{8}$$

F1-score is the harmonic mean of Precision and Recall. It provides a balanced measure that considers both false positives and false negatives. The harmonic mean is a type of average that gives more weight to smaller values in a dataset. It is calculated by dividing the number of values in the dataset by the sum of the reciprocals of the values. This means smaller values impact the harmonic mean more than more significant ones. The formula for F1-score is:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{9}$$

These metrics are closely related to each other. Accuracy provides an overall assessment of the system's correctness. At the same time, Precision and Recall focus on specific aspects of the predictions—Precision emphasises the accuracy of positive predictions, and recall highlights the system's ability to identify all positive instances. The F1-score balances Precision and Recall, providing a single metric that captures both aspects.

In the context of the evaluation results—where the average accuracy is 0.5742, average precision is 0.6689, average recall is 0.3245, and average F1-score is 0.4593—these values provide insights into the system's performance. The average accuracy indicates that approximately 57.42% of the recommendations made by the system were correct. The average precision of 0.6689 suggests that, on average, about 66.89% of the positive predictions made by the system were accurate. However, the average recall of 0.3245 implies that the system only identified around 32.45% of all positive instances. Consequently, the average F1-score of 0.4593 reflects the harmonic mean of Precision and Recall [42], offering a balanced assessment of the system's performance.

By analysing these evaluation metrics collectively, stakeholders can comprehensively understand the recommendation system's strengths and weaknesses, facilitating informed decision-making and potential improvements in its design and implementation.

5.2. Comparison of Different Model Configurations

Various configurations were explored to optimise the multiclass classification model and to identify the most effective setup. Given the problem's constraints—specifically, the requirement for the output activation functions to be SIGMOID or SOFTMAX—several combinations were tested to assess their performance.

One configuration attempted was setting the output activation function as SOFTMAX with the loss function CROSSENTHROPY, coupled with different choices for the hidden layer activation functions. Two configurations were tested: sigmoid(hidden) with softmax:ce(output) and relu(hidden) with softmax:ce(output). However, both configurations yielded unsatisfactory results, with average metrics falling below 0.3.

Undeterred by the initial unsatisfactory results, the exploration continued with additional configurations. One such approach involved introducing an extra sigmoid or relu hidden layer between the original configuration's layers. This iterative process of testing and refining was a testament to the researcher's persistence and dedication. However, despite these alterations, no significant improvements were observed in the model's performance.

In a final set of configuration attempts, the activation function tanh was introduced. Unlike sigmoid and relu, tanh is a hyperbolic tangent function that maps input values to the range [-1, 1] [43]. Mathematically, tanh(x) is defined as:

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \tag{10}$$

In this setup, the output activation function remained sigmoid with the loss function binary cross-entropy (BCE). Various combinations of tanh and other activation functions were tested in the hidden layers, such as tanh alone, relu + tanh, and tanh + tanh. While these configurations did not surpass the original setup in terms of performance metrics, they exhibited only a slight decrease in scores compared to the original configuration. This contrasted starkly with the configurations involving softmax activation, which resulted in a significant decline in performance metrics. This stark contrast in performance underscored the importance of the choice of activation functions, particularly for the hidden layers, in the model's overall performance.

These extensive comparisons showed that the choice of activation functions, particularly for the hidden layers, played a crucial role in the model's performance. Exploring different configurations provided valuable insights into the interplay between activation functions and model outcomes, paving the way for informed decisions in model optimisation.

5.3. Changing Epochs, Layer Size, and Learning Rate

In the optimization process of the neural network model, various parameters were adjusted and fine-tuned to enhance its performance. One critical parameter is the concept of epochs, which refers to the number of times the entire dataset is passed forward and backwards through the neural network during training [44].

Initially, the model was trained with 50 epochs. However, through experimentation, it was found that reducing the number of epochs to 10 or 25 led to lower scores. Conversely, increasing the epochs to 100 resulted in a 2% improvement in performance metrics. Nonetheless, it's important to note that higher epochs entail higher computation costs due to the increased dataset iteration.

Layer size is another significant factor influencing the model's performance [45]. Initially, the model consisted of five hidden layers. After trial and error, it was observed that setting the hidden layer size to 25 neurons yielded better results. However, the most optimal performance was achieved when the hidden layer size was increased to 100 neurons, indicating a more extensive presentation of the data.

The choice of learning rate optimization at all the newly adjusted settings also plays a crucial role in model performance [46]. One such algorithm is Nesterov's Accelerated Gradient (NAG), a Stochastic Gradient Descent (SGD) algorithm variant [47]. This method employs a momentum term to accelerate convergence and avoid oscillations. By setting the momentum parameter to 0.9, similar scores were obtained compared to the original model, indicating a resemblance in the optimization process between SGD and Nesterov's method.

Another optimization algorithm explored is Adam (Adaptive Moment Estimation), which combines the advantages of both Adagrad and RMSprop algorithms. Adam dynamically adjusts the learning rate for each parameter based on estimates of the first and second moments of the gradients [48]. The Adam algorithm exhibited significantly improved performance results at all the newly adjusted settings, as shown in Fig.9. This improvement can be attributed to Adam's adaptive nature, which adjusts the learning rate dynamically based on the gradient magnitudes and their history.

Upon evaluation, the model trained with Adam optimizer achieved an average accuracy of 0.9861, average precision of 0.8062, average recall of 0.7353, and average F1-score of 0.8314. These values represent substantial improvements compared to the previous model, with an accuracy of 0.5742, precision of 0.6689, recall of 0.3245, and F1-score of 0.4593. Furthermore, experimenting with different learning rates revealed that a rate of 0.0001 excluded more classes and led to lower scores, while a rate of 0.01 excluded fewer classes than the original but resulted in slightly lower performance metrics.

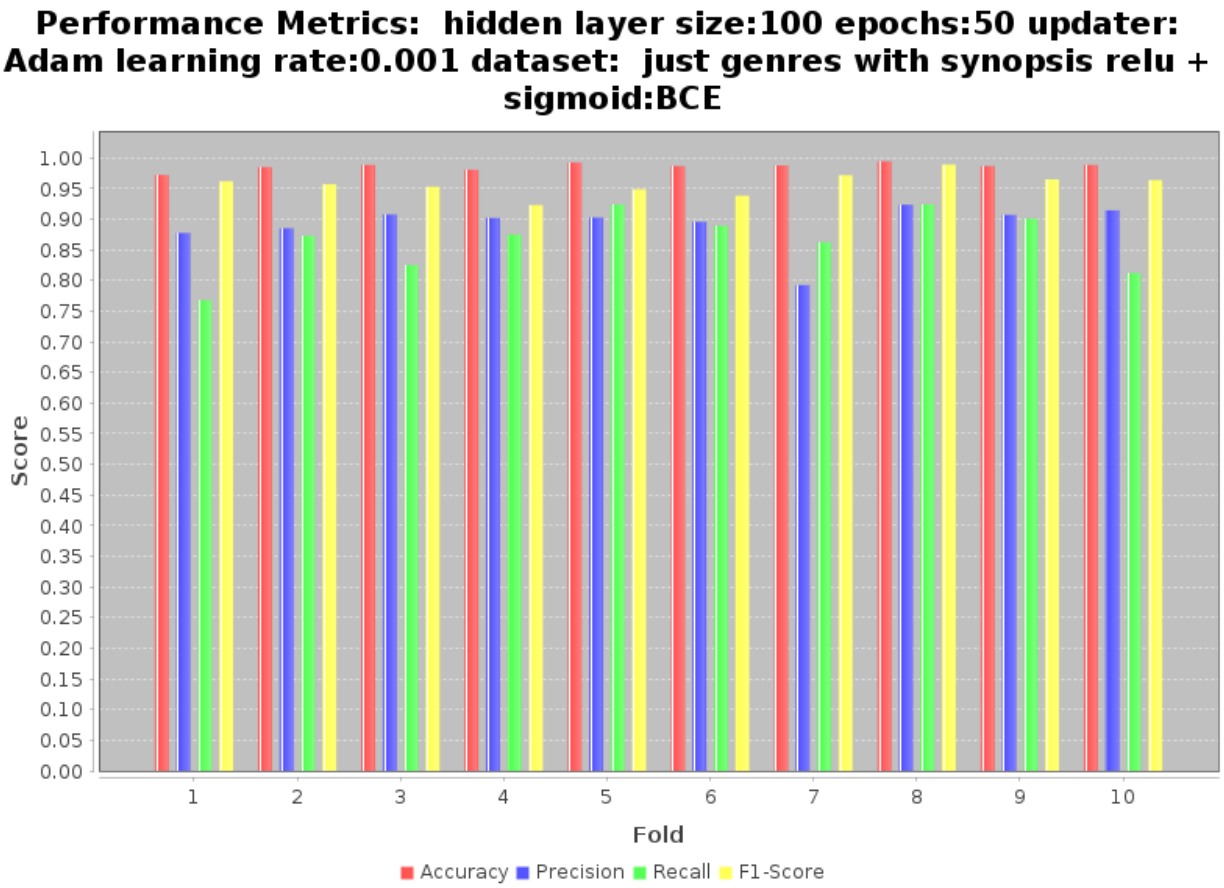


Fig 9: Evaluation metrics of neural network with Adam.

5.4. Exploration of Evaluation Techniques like ROC and AUC

5.5. Cross-validation vs. Train-test Split

Researchers often employ different techniques to gauge machine learning models' efficacy. Two prominent methods used for this purpose are k-fold cross-validation and train-test split. Each approach has advantages and limitations, which must be carefully considered based on the study's requirements.

K-fold cross-validation is a robust technique that partitions the dataset into k equally sized folds. In each iteration, one fold is designated as the validation set, while the remaining folds serve as the training data. This process is repeated k times, with each fold taking turns as the validation

set [49]. The performance metrics obtained from each iteration are then averaged to evaluate the model's performance comprehensively.

The main advantage of k-fold cross-validation is its ability to utilize the entire dataset for training and validation, thereby maximizing data utilization and reducing the risk of overfitting. However, this method can be computationally intensive, particularly for large datasets or complex models. Additionally, due to its increased computational overhead, it may not be suitable for time-sensitive applications.

In contrast, a train-test split involves dividing the dataset into two subsets: a training set used for model training and a test set used for evaluation. Typically, a percentage of the data, such as 80%, is allocated to the training set, while the remaining percentage is reserved for the test set [50]. This method is more straightforward and computationally efficient than k-fold cross-validation, making it suitable for quick model prototyping and exploration.

However, train-test split may lead to overfitting or underfitting, especially when the dataset is small or unbalanced. Additionally, the performance estimate obtained from a single split may be highly variable, depending on the randomness of the data partitioning.

The study examined the average scores obtained from both methods to compare the k-fold cross-validation and train-test split performance. Summarised results are in the table below:

TABLE 2
Comparison of Performance Metrics

Metric	**10-fold Cross-validation**	**Classes excluded**	**80:20 Split**	**Classes excluded**
Accuracy	0.9861	0	0.9841	0
Precision	0.8062	23	0.8542	37
Recall	0.7353	19	0.7838	35
F1 Score	0.8314	26	0.9451	41

From the comparison in Table 2, it is evident that the 80:20 train-test split resulted in higher average scores across most metrics compared to the 10-fold cross-validation method; however, more classes have been excluded from the average for the 80:20 split. Despite its higher computational cost, k-fold cross-validation provides more reliable and generalizable estimates of the model's performance, making it a preferred choice for rigorous evaluation.

5.6. Genre Reduction Using K-means++

Determining the optimal number of clusters in K-means++ clustering is a critical step to ensure the effectiveness of the genre amalgamation process [51]. One commonly used technique to ascertain this number is the elbow method. This method focuses on the within-cluster sum of squares (WCSS), which is the total squared distance between each point in a cluster and the centroid of that cluster [52]. Essentially, WCSS is a measure of the compactness of the clusters, with a lower value generally indicating a model with a better fit, as it suggests that the points are closer to their respective centroids.

The elbow method involves plotting the WCSS against the number of clusters. As the number of clusters increases, the WCSS naturally decreases because the clusters are smaller, and their centroids are closer to the points in the cluster. However, the rate of decrease will sharply drop at a certain point, forming an "elbow" shape in the plot. This point, often where further increases in clusters lead to only marginal improvements in WCSS, is considered the optimal number of clusters. The rationale behind this method is that adding more clusters beyond this point does not provide sufficient improvement in variance reduction to justify the additional complexity.

The simplicity and visual nature of the elbow method make it accessible and straightforward for determining the number of clusters. It provides a clear criterion based on a tangible metric (WCSS), making it easy to identify when additional clusters cease to add value. However, the method's primary drawback lies in its subjectivity—identifying the exact "elbow" point can be ambiguous and may vary depending on the individual analyzing the plot. Moreover, the elbow method can sometimes lead to inconsistent results, especially in datasets where the decrease in WCSS is relatively linear or when the proper structure of the data does not suggest a single elbow point.

Compared to more sophisticated methods like the silhouette score, which measures how similar an object is to its cluster compared to other clusters [53], the elbow method is less precise. The silhouette score provides a more nuanced view of cluster separation and cohesion, offering a numerical value that can help objectively decide the number of clusters. However, despite its potential for ambiguity and less precision relative to more complex methodologies, the elbow method remains popular due to its ease of implementation and straightforward interpretation, particularly in applications where a quick and reasonably accurate estimate of cluster numbers is sufficient. This is especially relevant in scenarios with strict time constraints, where the speed of the elbow method in providing a usable result can significantly expedite the decision-making process, allowing for faster progression in projects with limited time frames.

Visualizations, facilitated by the JFree Chart (Fig.10), reveal the attempt to determine the el-

bow point at 26 classes, leading to genre clusters as depicted in subsequent figures. These clusters exhibit varying degrees of success; for instance, the genres 'Space' and 'Military' converge into a single cluster, reflecting their thematic and contextual overlaps as seen in their word clouds in fig.11. Conversely, 'Space' distinctly separates from 'Slice of Life', indicating fewer shared terms (Fig.12).

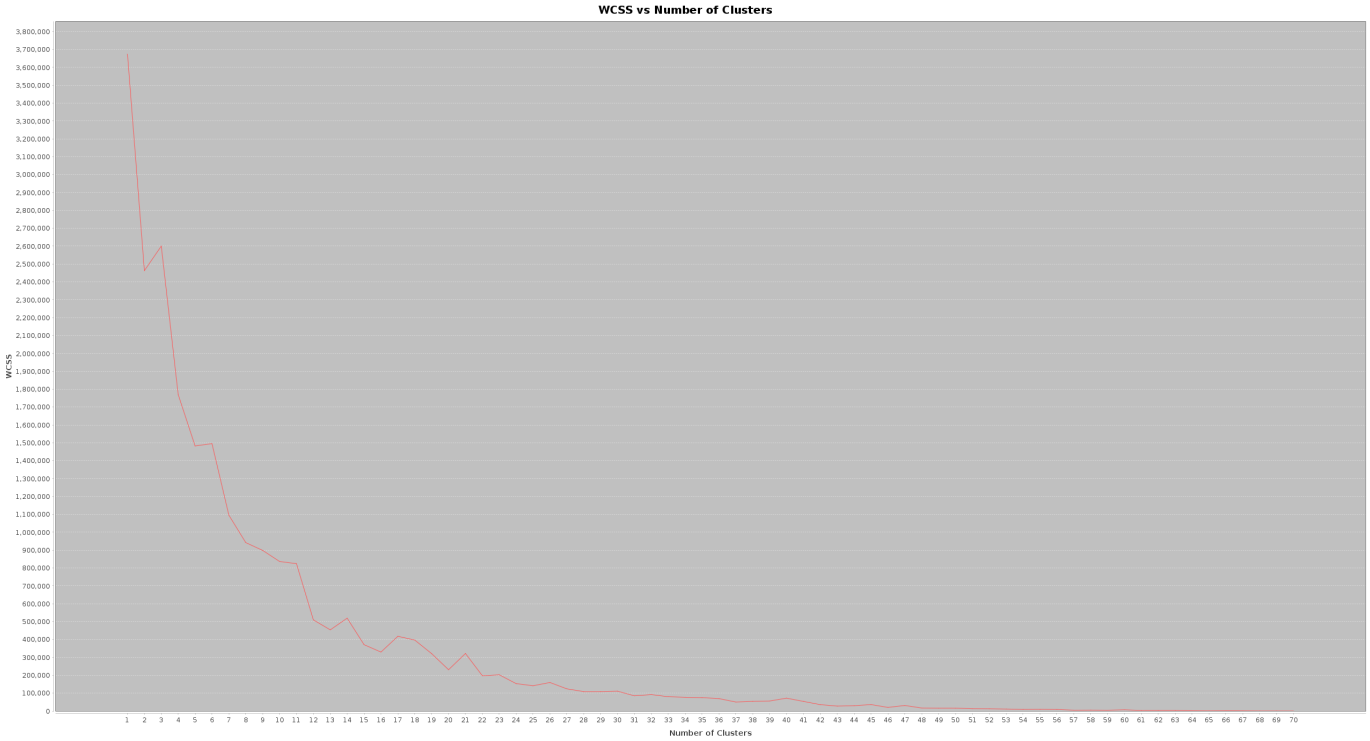
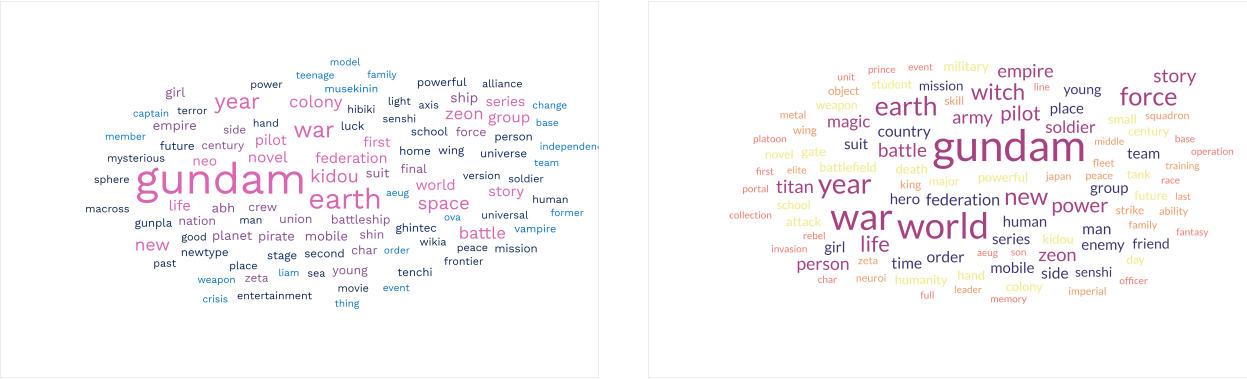


Fig 10: WCSS plot graph.

Identifying the elbow point, however, took much work. The WCSS plot graph displayed multiple peaks, which complicates the determination of the optimal number of clusters [54]. Multiple peaks in the WCSS graph suggest a dataset with several potential cluster solutions that could be reasonably justified, depending on the desired clustering granularity. This characteristic indicates a complex underlying structure in the data where distinct subgroups might be present within broader

categories, each potentially warranting separate consideration.

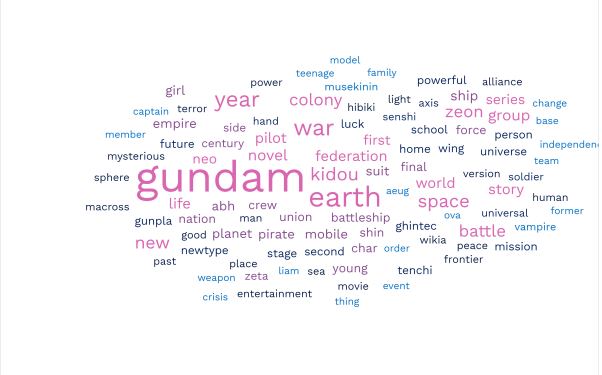
This complexity required careful analysis to choose an elbow point that best balances between too many clusters, which could overfit and obscure broader patterns, and too few, which might overlook subtle but meaningful distinctions between genres [55]. Such intricacies in the data underscore the challenges inherent in applying the elbow method, especially when clear demarcations between clusters are not evident, and they highlight the need for additional validation techniques or criteria to support the choice of an optimal number of clusters.



(a) Space **(b)** Military

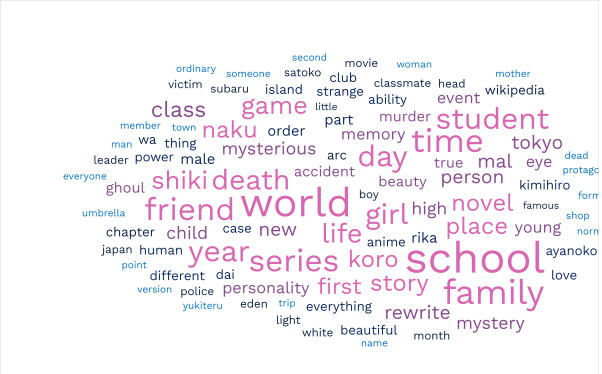
Fig 11: Similar Word Clouds

However, the clustering approach also surfaces limitations. For example, cluster number 2 amalgamates over 20 genres, while others contain only single or paired genres, as shown in Fig.14. This uneven distribution suggests an over-generalization for some genres and under-representation for others. Notably, genres like 'Psychological' and 'Parody' are grouped despite minimal commonalities beyond a few shared words, underscoring an inconsistency in a thematic grouping, as displayed in Fig.13. This observation challenges the reliability of using K-means++ clustering for genre-merging in this context, suggesting that while it has potential, it requires careful consideration and supplementary criteria to ensure meaningful genre integration.



(a) Space **(b)** Slice of Life

Fig 12: Differing Word Clouds in different clusters



(a) Psychological **(b)** Parody

Fig 13: Differing Word Clouds in the same cluster

```
Cluster 1 contains genres: [Adventure]
Cluster 2 contains genres: [Strategy Game, CGDCT, Otaku Culture,
  Avant Garde, Gore, Sports, Girls Love, Reverse Harem, Samurai,
  Showbiz, Combat Sports, Gag Humor, Workplace, Survival, Parody,
  Romantic Subtext, Team Sports, Medical, Iyashikei, Organized Crime,
  High Stakes Game, Erotica, Anthropomorphic, Mahou Shoujo,
  Psychological, Martial Arts, Crossdressing, Childcare, Idols (Male),
  Suspense, Visual Arts, Idols (Female), Video Game, Love Polygon, Seinen,
  Adult Cast, Detective, Performing Arts, Magical Sex Shift, Boys Love,
  Music, Time Travel, Gourmet, Historical]
Cluster 3 contains genres: [Comedy]
Cluster 4 contains genres: [Isekai, Reincarnation]
Cluster 5 contains genres: [Shoujo]
Cluster 6 contains genres: [Josei]
Cluster 7 contains genres: [Fantasy]
Cluster 8 contains genres: [Hentai]
Cluster 9 contains genres: [Drama]
Cluster 10 contains genres: [Harem]
Cluster 11 contains genres: [Mecha]
Cluster 12 contains genres: [School]
Cluster 13 contains genres: [Shounen]
Cluster 14 contains genres: [Romance]
Cluster 15 contains genres: [Supernatural]
Cluster 16 contains genres: [Action]
Cluster 17 contains genres: [Mythology]
Cluster 18 contains genres: [Slice of Life]
Cluster 19 contains genres: [Space, Military]
Cluster 20 contains genres: [Sci-Fi]
Cluster 21 contains genres: [Super Power]
Cluster 22 contains genres: [Horror]
Cluster 23 contains genres: [Villainess]
Cluster 24 contains genres: [Mystery]
Cluster 25 contains genres: [Vampire]
Cluster 26 contains genres: [Ecchi]
```

Fig 14: Clustering Results using K-means++.

6. Conclusions and Future Work

6.1. Summary of Main Results and Achievements

The study successfully developed and evaluated a recommendation system based on a hybrid approach combining collaborative and content-based filtering techniques. The system has demonstrated promising performance in recommending relevant items to users through rigorous experimentation and evaluation.

6.2. Critique of Achievements concerning Project Goals

The achievements of the recommendation system align well with the project goals and objectives. The system effectively addresses the need to provide users with personalised recommendations based on their preferences and historical interactions with the platform. Furthermore, the hybrid approach adopted in the system allows for leveraging user-item interactions and item content information, enhancing recommendation accuracy and coverage. However, it is essential to note that there were certain areas where the system fell short. Firstly, it lacked a user-friendly UI, which could have elevated the user experience and facilitated easier interaction with the system. Additionally, incorporating feedback from other users could have provided valuable insights into the recommendations' effectiveness and helped refine the recommendation algorithms.

Furthermore, the system encountered challenges in successfully reducing the dimensionality of classes when adding words. Although the exploration of word2vec, a method for generating word embeddings, was considered, it needed to be implemented in the development process to fully take advantage of its benefits. These areas represent potential avenues for future improvement and optimisation of the recommendation system.

6.3. Identification of Difficulties, Limitations, and Lessons Learned

Despite significant strides, the recommendation system encountered several challenges and limitations during its development and evaluation. One prominent issue was the presence of data sparsity, which hindered the system's ability to make accurate recommendations, particularly in niche areas. However, the system effectively handled cold start problems for new items by automatically integrating them into the dataset whenever updates occurred.

Scalability concerns were not a major obstacle, but the quality of data notably influenced the recommendation system's performance. The lack of comprehensive synopses for most light novels proved to be a significant challenge, directly affecting the system's ability to extract meaningful features for recommendation.

The project also faced limitations in terms of time constraints and API stability. Limited time availability, compounded by the need to allocate resources to other project modules, hampered the system's development progress. Additionally, occasional API instability caused disruptions, requiring time and effort.

From these challenges, valuable lessons have been learned. It became evident that practical feature engineering plays a crucial role in the performance of machine learning algorithms [56], underscoring the importance of investing time and effort into this aspect of model development. Furthermore, the experience highlighted the necessity of adequately estimating the training time required for neural networks and exploring multiple algorithms for various network components, such as loss functions and updater algorithms. Additionally, the vast array of methods available for evaluating and displaying performance metrics necessitates careful consideration and experimentation to identify the most effective approaches. These lessons will inform future endeavours in refining and optimizing the recommendation system.

6.4. Suggestions for Further Work and Extensions to the Project

There are several avenues for future work and extensions to enhance the recommendation system. Firstly, improving the quality of word embeddings, such as employing techniques like word2vec, would enable more efficient processing of synopses and other textual data alongside genres within the neural network. This enhancement could lead to more accurate and contextually relevant recommendations.

Additionally, enhancing the user interface with a simple yet intuitive design [57] would facilitate easier visualization of recommendations through images and similarity scores. A visually appealing and user-friendly interface can significantly enhance the user experience, encouraging greater engagement and satisfaction with the recommendation system.

Furthermore, incorporating sentiment analysis and advanced natural language processing techniques could provide deeper insights into user preferences and content characteristics. The recommendation system can offer more personalised and contextually relevant recommendations by analyzing the sentiment and context of user interactions and content descriptions, thereby improving user satisfaction and engagement.

Moreover, exploring hybridization with other recommendation approaches, such as knowledge-based filtering and reinforcement learning-based approaches, could unlock new insights and opportunities for enhancing recommendation performance. By combining different recommendation techniques, the system can leverage the strengths of each approach to provide more accurate and diverse recommendations tailored to individual user preferences and needs.

Finally, conducting comprehensive user studies and real-world evaluations will be essential for validating the effectiveness and usability of the recommendation system in practical settings. Gathering user feedback and evaluating the system's performance in real-world scenarios can provide valuable insights into its strengths, weaknesses, and areas for improvement, guiding future development efforts.

Overall, the recommendation system presents a promising foundation for future research and development efforts to advance personalized recommendation technologies and deliver enhanced user experiences.

7. Author's Assessment of the Project

- **Relevance**. Delved into classification using neural networks, explored various algorithms for classification, and implemented a hybridized approach, showing its superiority over single methods.
- **Utilization** The model is not just a theoretical concept. It is a practical tool that allows others to contribute text-based entries for the model to train on. They can classify their data through transfer learning and use content-based filtering to recommend similar entries.
- **Achievement**: Successfully created a dataset with 10k values, achieved desired accuracies through rigorous hyperparameter fine-tuning, and obtained recommendations based on user input.
- **Limitations**: It is limited by time constraints, relies on APIs for initial data accuracy, cannot handle large label quantities due to computational costs, and needs a UI for visualizing recommendations; it only displays text data in the command line.

8. Project Management

8.1. Project Management Practices

It is essential to delineate the critical practices that serve as cornerstones in ensuring the success of any project. Among these, several foundational practices emerge as pivotal elements of effective project management.

While continuous monitoring and control of project progress are essential practices in project management, their full extent has yet to be realized in this individual project. Although project progress has been monitored through supervisor meetings and calls, the monitoring should have been more continuous and comprehensive. Constant monitoring allows for the vigilant tracking of key performance indicators, milestones, and deliverables, enabling early identification of deviations

from the plan. This proactive approach facilitates timely corrective actions, mitigating risks and maintaining project momentum effectively, safeguarding project success.

Furthermore, utilizing project management tools is crucial for enhancing project management practices. These tools offer task management, scheduling, document sharing, and reporting features, facilitating efficient collaboration among individual members regardless of their physical locations [58]. Despite their importance, this project has yet to utilize the full range of project management tools. These tools empower project managers to optimize individual productivity, track real-time progress, and make informed decisions to drive project success. Thus, incorporating comprehensive project management tools would have facilitated better project management and monitoring throughout the project lifecycle.

8.2. Objectives and Goals

For this project, several clear objectives and goals were established to guide its execution:

- Produce a comprehensive dataset comprising more than 10,000 light novel entries. Each entry should include essential information such as ID, name, authors, genres, synopsis, and alternative names. This dataset serves as the foundational resource for subsequent analysis and modelling tasks.
- Implement a neural network architecture with two layers, including one hidden layer. The neural network should be capable of effectively learning from the provided dataset and making accurate predictions or classifications.
- Utilize natural language processing techniques to vectorize the synopsis data within the dataset. By transforming textual information into numerical representations, the vectorization process aims to enable efficient processing and analysis of the synopsis content.
- Reduce the dimensionality of the dataset by limiting the number of classes to 100. This reduction in class complexity facilitates model training and improves computational efficiency while preserving meaningful distinctions between entries.
- Set a performance benchmark of achieving an accuracy rate of 80% or above for the neural network model. This metric quantitatively measures the model's effectiveness in accurately predicting or classifying light novel entries based on the provided features.
- Implement a system for relabeling the entries within the dataset to facilitate content-based filtering. By assigning relevant labels or classifications to each entry, the dataset becomes more organized and accessible, enabling recommendation systems and other filtering mechanisms to be implemented.

By delineating these clear objectives and goals, the project aims to ensure focused efforts, efficient

resource allocation, and the successful achievement of its intended outcomes.

8.3. Supervisor Meetings

Over the academic year, 15 supervisor meetings were convened, serving as pivotal touchpoints in the research journey.

Most of these meetings were conducted in person, providing a conducive environment for in-depth discussions and meaningful exchanges of ideas. However, due to logistical constraints and scheduling conflicts, a few meetings were conducted via video conference calls, ensuring continuity and accessibility despite physical distance.

Central to the agenda of these meetings was a comprehensive review of progress made since the previous session. Through detailed presentations and discussions, key findings, challenges encountered, and potential bottlenecks were thoroughly examined, fostering a deeper understanding of the research landscape.

Moreover, these meetings were forums for exploring the following steps and charting the course. Supervisors provided invaluable guidance, insights, and recommendations, drawing upon their expertise and experience to steer the research in the right direction. Emphasis was placed on the importance of rigorous data analysis and understanding before implementation, underscoring the need for a robust foundation to support subsequent research endeavours.

Supervisor meetings played a pivotal role in guiding and shaping the trajectory of the research project. By facilitating open dialogue, critical reflection, and strategic planning, these sessions enabled researchers to navigate challenges, capitalize on opportunities, and advance towards attaining research objectives. In the subsequent sections, we will delve deeper into the outcomes and insights from these meetings, elucidating their impact on the research process and outcomes.

8.4. Challenges faced

In pursuing academic research, it is inevitable to encounter various challenges and obstacles that may impede progress and necessitate adaptation and resilience. Several challenges were faced throughout this project, each requiring careful navigation and mitigation strategies.

In January, a significant setback occurred due to a period of heavy illness experienced by the researcher. This unforeseen health issue significantly hindered progress and productivity, necessitating a period of recuperation and convalescence. Consequently, valuable time and momentum were lost, necessitating diligent efforts to compensate for lost ground in subsequent months.

In February 2024, an unexpected disruption occurred when the MyAnimeList API experienced a technical malfunction, rendering it inaccessible for a week. This unforeseen outage posed a signif-

icant challenge regarding data acquisition, disrupting the planned workflow and slowing progress. However, this period of downtime was utilized effectively by preparing and refining code for data analysis, ensuring that productivity was maintained despite the temporary setback. As a result, backup datasets were created weekly to prevent further setbacks.

Furthermore, a slight deceleration in progress was observed due to the time required to understand the intricacies of the dataset and gain proficiency in coding neural networks in Java. The complexity of the data and the technical nuances of neural network implementation posed formidable challenges, requiring dedicated effort and time to overcome. However, the researcher's commitment to continuous learning was evident. These challenges were gradually surmounted through perseverance and continuous learning, enabling progress to resume steadily and highlighting the researcher's dedication to improving their skills and knowledge.

The challenge of effective time management also emerged as a recurring issue throughout the project. Balancing personal commitments, academic responsibilities, and the demands of other modules proved to be a formidable task, often leading to competing priorities and time constraints. As a result, optimal allocation of time and resources became paramount, necessitating strategic planning and prioritization to ensure that project milestones were met promptly.

In summary, the academic research journey is fraught with challenges and obstacles that demand resilience, adaptability, and perseverance. By acknowledging and addressing these challenges head-on, researchers can navigate adversity, overcome setbacks, and ultimately emerge more substantial and resilient in pursuing knowledge and innovation.

8.5. Timetable

The project entails developing a detailed Gantt chart, provided in appendix C, produced with the help of online software [59], meticulously outlining the project's timeline and milestones. This chart is invaluable for monitoring progress and ensuring adherence to established timelines and objectives. Given the individual nature of the project, the Gantt chart provides a visual representation of the sequential activities and resource allocations required for project completion [60]. Through the systematic tracking of tasks and milestones, the Gantt chart enables effective time management and aids in identifying potential bottlenecks or delays. Utilizing this structured approach to project management, the individual can proactively address challenges, allocate resources judiciously, and maintain a focused trajectory towards achieving project goals.

According to the Gantt chart, a user interface (UI) component was scheduled for completion within the last few weeks. However, unforeseen challenges, such as illness, API disruptions, and time constraints, have led to setbacks in the development schedule. As a result, the UI implementation

has been delayed, necessitating a reassessment of project priorities. Given the impending deadline, the focus has shifted towards prioritizing tasks essential to achieving high accuracies, such as fine-tuning and implementing the hybrid approach. By reallocating resources and efforts towards these critical components, the project aims to maximize its chances of meeting its objectives within the available timeframe.

9. Ethical and Legal Considerations

The project is exempt from ethical consent requirements, as it does not entail using user data, alleviating potential privacy and confidentiality concerns. The MyAnimeList API, from which data is sourced, is readily accessible under its respective documentation and is intended solely for educational purposes [61]. Furthermore, the data retrieved from this API, which is publicly available, does not contain any sensitive or personally identifiable information.

Moreover, the libraries integrated into the project framework are open-source and adhere to licensing terms that permit unrestricted use for academic and research purposes. Consequently, ethical and legal considerations about the use of these libraries are adequately addressed, ensuring compliance with ethical standards and regulations governing academic research endeavours.

10. Acknowledgements

The successful completion of this project would not have been possible without the invaluable contributions and support of several individuals and communities, to whom I extend my sincere gratitude.

First and foremost, I would like to express my heartfelt appreciation to Ayse Saliha Sunar for her unwavering support and guidance throughout the project. Her invaluable insights into data visualization and her continuous efforts in organizing and facilitating meetings were instrumental in keeping the project on track and ensuring its success.

I would also extend my gratitude to the MyAnimeList API discord community for their invaluable assistance and support. Their prompt responses to queries related to the API and their diligent efforts in informing the community about API disruptions were immensely helpful in navigating technical challenges and maintaining project momentum.

Additionally, I am deeply indebted to Florin Ciucu for his invaluable mentorship and guidance in critical thinking. His insightful inputs and constructive feedback were instrumental in shaping my approach to genre-merging and enhancing the overall quality of the project.

Finally, I express my gratitude to all those who provided moral support and encouragement throughout the project. Your unwavering belief in my abilities and encouragement were a constant

source of motivation and inspiration.

In conclusion, I am profoundly grateful to each individual and community mentioned above for their invaluable contributions and unwavering support. Their collective efforts have been instrumental in completing this project, and their contributions will be forever cherished and remembered.

References

[1] Andrea Crook. *How AI is Revolutionizing E-commerce and Online Shopping*. Digital Doughtnut. Aug. 2023. URL: https://www.digitaldoughnut.com/articles/2023/july-2023/how-ai-is-revolutionizing-ecommerce (visited on 04/30/2024).

[2] ai-admin. *The Power and Promise of AI Analytics - Unleashing the Potential of Data-driven Insights*. AI for Social Good. Dec. 2023. URL: https://aiforsocialgood.ca/blog/the-power-and-promise-of-ai-analytics-unleashing-the-potential-of-data-driven-insights (visited on 04/30/2024).

[3] P. B. Thorat, R. M. Goudar, and S. Barve. "Survey on collaborative filtering, content-based filtering and hybrid recommendation system". In: *International Journal of Computer Applications* (2015). URL: https://www.academia.edu/download/59762468/10.1.1.695.642820190617-91457-z4s1rf.pdf (visited on 04/30/2024).

[4] Biliana Lika, Kostas Kolomvatsos, and Stathes Hadjiefthymiades. "Facing the cold start problem in recommender systems". In: *Expert Systems with Applications* 41.4, Part 2 (2014), pp. 2065–2073. DOI: 10.1016/j.eswa.2013.09.007.

[5] Guangneng Guo. "Resolving data sparsity and cold start in recommender systems". In: *User Modeling, Adaptation, and Personalization: 20th International Conference, UMAP 2012*. Berlin, Heidelberg: Springer, 2012, pp. 342–353. DOI: 10.1007/978-3-642-31454-4_36.

[6] Dominik Kowald et al. "A Study on Accuracy, Miscalibration, and Popularity Bias in Recommendations". In: *arXiv preprint arXiv:2303.00400* (2023). URL: https://arxiv.org/abs/2303.00400v1.

[7] Reem Alabdulrahman and Herna Viktor. "Catering for unique tastes: Targeting grey-sheep users recommender systems through one-class machine learning". In: *Expert Systems with Applications* 168 (2021), p. 114366. DOI: 10.1016/j.eswa.2020.114366.

[8] Debajyoti Roy and Mainak Dutta. "A systematic review and research perspective on recommender systems". In: *Journal of Big Data* 9 (2022), p. 12. DOI: 10.1186/s40537-022-00592-5.

[9] Taro Fujimoto. "The evolution of light novels and their popularity in Japan". In: *Journal of Japanese Studies* 39.1 (2013), pp. 111–130. DOI: 10.1353/jjs.2013.0006.

[10] Sarah K. Manner. “Behind every great manga: The rising popularity of Japanese light novels”. In: *Florida Libraries* 59.2 (2016), pp. 44–48.

[11] *Light Novel Recommender — AniBrain*. AniBrain. URL: https://anibrain.ai/recommender/light-novel (visited on 04/29/2024).

[12] Tulasi K. Paradarami, Nathaniel D. Bastian, and Jennifer L. Wightman. “A hybrid recommender system using artificial neural networks”. In: *Expert Systems with Applications* 83 (Oct. 2017), pp. 300–313. DOI: 10.1016/j.eswa.2017.04.046. URL: https://dx.doi.org/10.1016/j.eswa.2017.04.046.

[13] Hao Wang, Naiyan Wang, and Dit-Yan Yeung. “Collaborative Deep Learning for Recommender Systems”. In: *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Aug. 2015), pp. 1235–1244. DOI: 10.1145/2783258.2783273. URL: https://arxiv.org/pdf/1409.2944 (visited on 03/28/2022).

[14] Houchou Han et al. “LFDNN: A Novel Hybrid Recommendation Model Based on DeepFM and LightGBM”. In: *Entropy* 25 (Apr. 2023), p. 638. DOI: 10.3390/e25040638. URL: https://www.mdpi.com/1099-4300/25/4/638.

[15] Heng-Tze Cheng et al. “Wide & Deep Learning for Recommender Systems”. In: *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*. June 2016, pp. 7–10. URL: https://arxiv.org/pdf/1606.07792 (visited on 04/29/2024).

[16] Paul Covington, Jay Adams, and Emre Sargin. “Deep Neural Networks for YouTube Recommendations”. In: *Proceedings of the 10th ACM Conference on Recommender Systems* RecSys ’16 (Sept. 2016), pp. 191–198. DOI: 10.1145/2959100.2959190. URL: https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45530.pdf (visited on 04/29/2024).

[17] P Devika, R C Jisha, and G P Sajeev. “A novel approach for book recommendation systems”. In: *2016 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC)*. 2016, pp. 1–6. DOI: 10.1109/ICCIC.2016.7919606. URL: https://dx.doi.org/10.1109/ICCIC.2016.7919606.

[18] Paula Cristina Vaz et al. “Improving an Hybrid Literary Book Recommendation System through Author Ranking”. In: *arXiv (Cornell University)* (Mar. 2012). DOI: 10.48550/arxiv.1203.5324. URL: https://arxiv.org/abs/1203.5324 (visited on 04/29/2024).

[19] S. S. Sohail, J. Siddiqui, and R. Ali. “Book Recommendation System UsingOpinion Mining Technique”. In: *2013 International Conference on Advances in Computing,Communications and Informatics (ICACCI)* (Aug. 2013). DOI: 10.1109/ICACCI.2013.6637421. (Visited on 10/23/2023).

[20] P. Malhotra, S. Rajpurkar, and D. Bhatt. "Book Recommendation System". In: *International Journal for Innovative Research in Science and Technology* 1.11 (2015), pp. 314–316.

[21] Praveena Mathew, Bincy Kuriakose, and Vinayak Hegde. "Book Recommendation System through content based and collaborative filtering method". In: *2015 International Conference on Soft-Computing and Networks Security*. Mar. 2015, pp. 47–52. DOI: 10.1109/SAPIENCE.2016.7684166.

[22] M. Naumov et al. "Deep Learning Recommendation Model for Personalization and Recommendation Systems". In: *arXiv preprint* (2019). eprint: arXiv:1906.00091.

[23] Xinxi Wang and Ye Wang. "Improving Content-based and Hybrid Music Recommendation using Deep Learning". In: *Proceedings of the ACM International Conference on Multimedia - MM '14* (2014). DOI: 10.1145/2647868.2654940. URL: https://smcnus.comp.nus.edu.sg/wp-content/uploads/2013/09/deep_mr.pdf.

[24] A. Van Den Oord, S. Dieleman, and B. Schrauwen. "Deep Content-Based Music Recommendation". In: *Advances in Neural Information Processing Systems*. 2013, pp. 2643–2651.

[25] Yassine Afoudi, Mohamed Lazaar, and Mohammed Al Achhab. "Hybrid recommendation system combined content-based filtering and collaborative prediction using artificial neural network". In: *Simulation Modelling Practice and Theory* 113 (July 2021), p. 102375. DOI: 10.1016/j.simpat.2021.102375.

[26] Warren S. McCulloch and Walter Pitts. "A Logical Calculus of the Ideas Immanent in Nervous Activity". In: *Bulletin of Mathematical Biophysics* 5.4 (1943), pp. 115–133.

[27] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep Learning". In: *Nature* 521.7553 (2015), pp. 436–444.

[28] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. "Deep Learning". In: *MIT Press* (2016).

[29] Y. Jin et al. "Fusion of Audio and Vibration Signals for Bearing Fault Diagnosis Based on a Quadratic Convolution Neural Network". In: *Sensors* 23.22 (2023), p. 9155.

[30] Marko Balabanović and Yoav Shoham. "Fab: content-based, collaborative recommendation". In: *Communications of the ACM* 40.3 (1997), pp. 66–72. DOI: 10.1145/245108.245124. URL: https://dl.acm.org/doi/pdf/10.1145/245108.245124.

[31] V. Cutting and N. Stephen. "Comparative Review of Java and Python". In: *International Journal of Research and Development in Applied Science and Engineering* (2021). URL: http://ijrdase.com/ijrdase/wp-content/uploads/2022/03/Comparative-Review-of-JAVA-and-Python.pdf.

[32] *Natural Language Inference Dataset for Software Engineering*. Zenodo. 2023. URL: https://doi.org/10.5281/zenodo.8025053 (visited on 04/29/2023).

[33] MyAnimeList.net. *Manga Ranking API*. 2023. URL: https://myanimelist.net/apiconfig/references/api/v2#operation/manga_ranking_get (visited on 05/01/2023).

[34] G. A. Nys et al. *A Semantic Retrieval System for Remote Sensing Web Platforms*. 2019. URL: https://orbi.uliege.be/bitstream/2268/236943/1/NYS_ISPRSSymposium.pdf (visited on 05/01/2023).

[35] Vinija's Notes. *Natural Language Processing • Word Vectors/Embeddings*. 2023. URL: https://vinija.ai/nlp/word-vectors/ (visited on 05/01/2023).

[36] W. Ruan, Y. Chen, and B. Forouraghi. "On Development of Data Science and Machine Learning Applications in Databricks". In: *Lecture Notes in Computer Science*. 2019. DOI: 10.1007/978-3-030-23381-5_6. URL: https://doi.org/10.1007/978-3-030-23381-5_6.

[37] Qian Song, Wei Luo, and Wei Luo. "SFBKT: A Synthetically Forgetting Behavior Method for Knowledge Tracing". In: *Applied Sciences* 13.13 (2023), p. 7704. DOI: 10.3390/app13137704.

[38] D. Liu, X. Chen, and D. Peng. "The intuitionistic fuzzy linguistic cosine similarity measure and its application in pattern recognition". In: *Complexity* 2018 (2018). DOI: 10.1155/2018/9073597. URL: https://www.hindawi.com/journals/complexity/2018/9073597/.

[39] Li Chia-Chun et al. "Duck Eggshell Crack Detection by Nondestructive Sonic Measurement and Analysis". In: *Sensors* 21.21 (2021), p. 7299. DOI: 10.3390/s21217299.

[40] Hsiu Mei Chou, Hsiu Mei Chou, and Tsai-Lun Chung. "Utilizing Text Mining for Labeling Training Models from Futures Corpus in Generative AI". In: *Applied Sciences* 13.17 (2023), p. 9622. DOI: 10.3390/app13179622.

[41] Zia Aziz and Robert Bestak. "Insight into Anomaly Detection and Prediction and Mobile Network Security Enhancement Leveraging K-Means Clustering on Call Detail Records". In: *Sensors* 24.6 (2024), p. 1716. DOI: 10.3390/s24061716.

[42] Jie Xiong et al. "Siamese Transformer-Based Building Change Detection in Remote Sensing Images". In: *Sensors* 24.4 (2024), p. 1268. DOI: 10.3390/s24041268.

[43] S. Sharma, S. Sharma, and A. Athaiya. "Activation functions in neural networks". In: *Towards Data Sci, IJEAST* (2017). URL: https://www.ijeast.com/papers/310-316,Tesma412,IJEAST.pdf.

[44] Jason Brownlee. "Gentle Introduction to the Concept of Epochs in Machine Learning". In: *Machine Learning Mastery* (2019). URL: https://machinelearningmastery.com/difference-between-a-batch-and-an-epoch/.

[45] D. Nguyen and B. Widrow. "Improving the Learning Speed of 2-Layer Neural Networks by Choosing Initial Values of the Adaptive Weights". In: *Proceedings of the International Joint Conference on Neural Networks* (1990), pp. 21–26. URL: https://ieeexplore.ieee.org/document/5726777.

[46] Leslie N. Smith. "Cyclical Learning Rates for Training Neural Networks". In: *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)* (2017), pp. 464–472. URL: https://ieeexplore.ieee.org/document/7926641/.

[47] X. Liu, W. Tao, and Z. Pan. "A Convergence Analysis of Nesterov's Accelerated Gradient Method in Training Deep Linear Neural Networks". In: *Information Sciences* 585 (2022), pp. 99–114. URL: https://www.sciencedirect.com/science/article/pii/S0020025522010003.

[48] Tianwei Xie and Yingxue Li. "Efficient Integer Vector Homomorphic Encryption Using Deep Learning for Neural Networks". In: *Lecture Notes in Computer Science*. 2018. DOI: 10.1007/978-3-030-04167-0_8. URL: https://doi.org/10.1007/978-3-030-04167-0_8.

[49] Davide Anguita et al. "K-Fold Cross Validation for Error Rate Estimate in Support Vector Machines". In: *DMIN*. 2009. URL: https://www.researchgate.net/publication/220704948_K-Fold_Cross_Validation_for_Error_Rate_Estimate_in_Support_Vector_Machines.

[50] Attila Rácz, Dávid Bajusz, and Károly Héberger. "Effect of dataset size and train/test split ratios in QSAR/QSPR multiclass classification". In: *Molecules* 26.4 (2021), p. 1111. DOI: 10.3390/molecules26041111. URL: https://www.mdpi.com/1420-3049/26/4/1111.

[51] Ablison. *Pros and Cons of K Means Clustering*. URL: https://www.ablison.com/pros-and-cons-of-k-means-clustering/ (visited on 04/30/2023).

[52] Michael J. Brusco and Douglas Steinley. "A comparison of heuristic procedures for minimum within-cluster sums of squares partitioning". In: *Psychometrika* 72.4 (2007), pp. 583–600. DOI: 10.1007/s11336-007-9013-4.

[53] F. Aksan et al. "Clustering Methods for Power Quality Measurements in Virtual Power Plant". In: *Energies* 14.18 (2021), p. 5902. DOI: 10.3390/en14185902.

[54] Chao Shi et al. "A quantitative discriminant method of elbow point for the optimal number of clusters in clustering algorithm". In: *Eurasip Journal on Wireless Communications and Networking* (2021). DOI: 10.1186/s13638-021-01910-w. URL: https://link.springer.com/article/10.1186/s13638-021-01910-w.

[55] Deepak Agrawal and Sushil Kushwaha. "Centriod Selection Process Using WCSS and Elbow Method for K-Mean Clustering Algorithm in Data Mining". In: *ijmemr* (2023). URL: http://ijmemr.org/Publication/V6I1/IJMEMR-V6I1-006.pdf.

[56] J. Kruse et al. "Leveraging the Sensitivity of Plants with Deep Learning to Recognize Human Emotions". In: *Sensors* 24.6 (2024), p. 1917. DOI: 10.3390/s24061917.

[57] *Strategies for Attracting and Retaining Players in Online Casinos — Mum Heat*. Mum Heat. Sept. 2023. URL: https://mumheat.com/strategies-for-attracting-and-retaining-players-in-online-casinos/ (visited on 04/30/2024).

[58] Muhammad Sajad, Muhammad Sadiq, and Khurram Naveed. "Software Project Management: Tools assessment, Comparison and suggestions for future development". In: *International Journal of Computer Science and Information Security* 14.2 (2016), pp. 560–569. URL: https://www.researchgate.net/publication/295080231_Software_Project_Management_Tools_assessment_Comparison_and_suggestions_for_future_development (visited on 05/01/2023).

[59] *Free Online Gantt Chart Software*. www.onlinegantt.com. URL: http://www.onlinegantt.com/ (visited on 04/27/2024).

[60] Productroadmap.ai. *What is the Purpose of a Gantt Chart and How is it Used in Project Management*. 2023. URL: https://www.productroadmap.ai/gantt/what-is-the-purpose-of-a-gantt-chart-and-how-is-it-used-in-project-management (visited on 04/29/2024).

[61] *API License and Developer Agreement*. myanimelist.net. URL: https://myanimelist.net/static/apiagreement.html (visited on 04/29/2024).

Appendix A: Genre Rating Graph

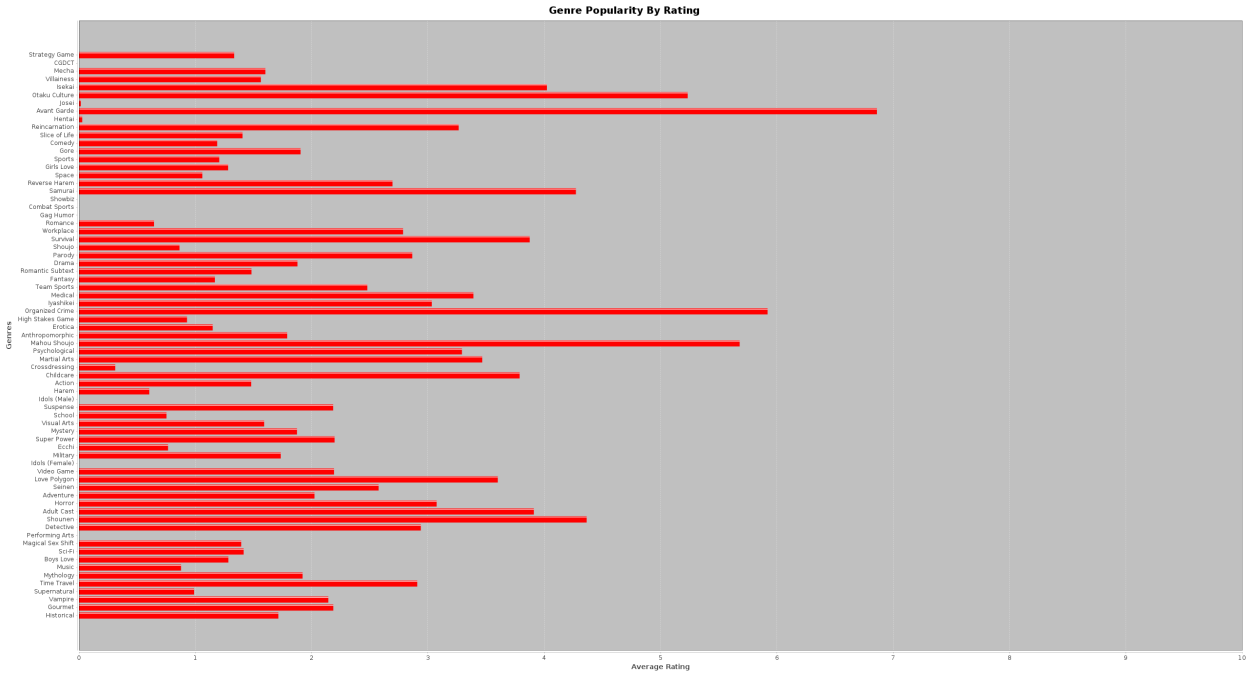


Fig 15: Genre Ratings.

Appendix B: Genre Distribution Graph

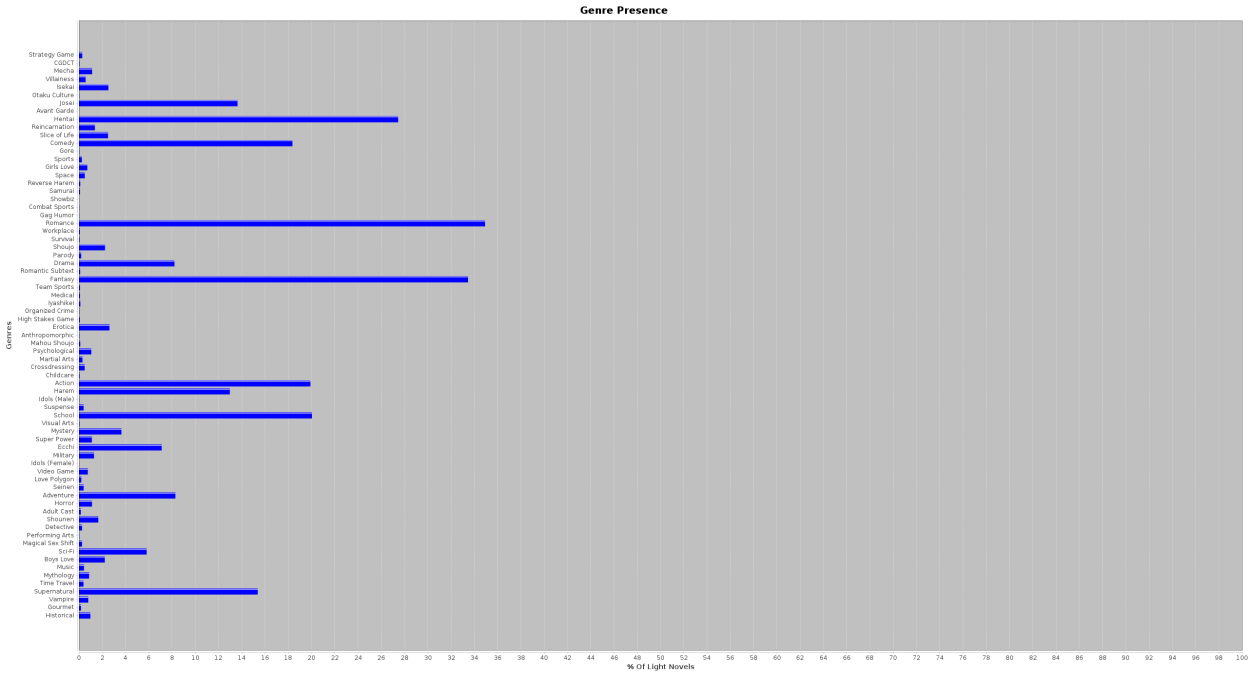


Fig 16: Genre Distribution.

Appendix C: Gantt Chart

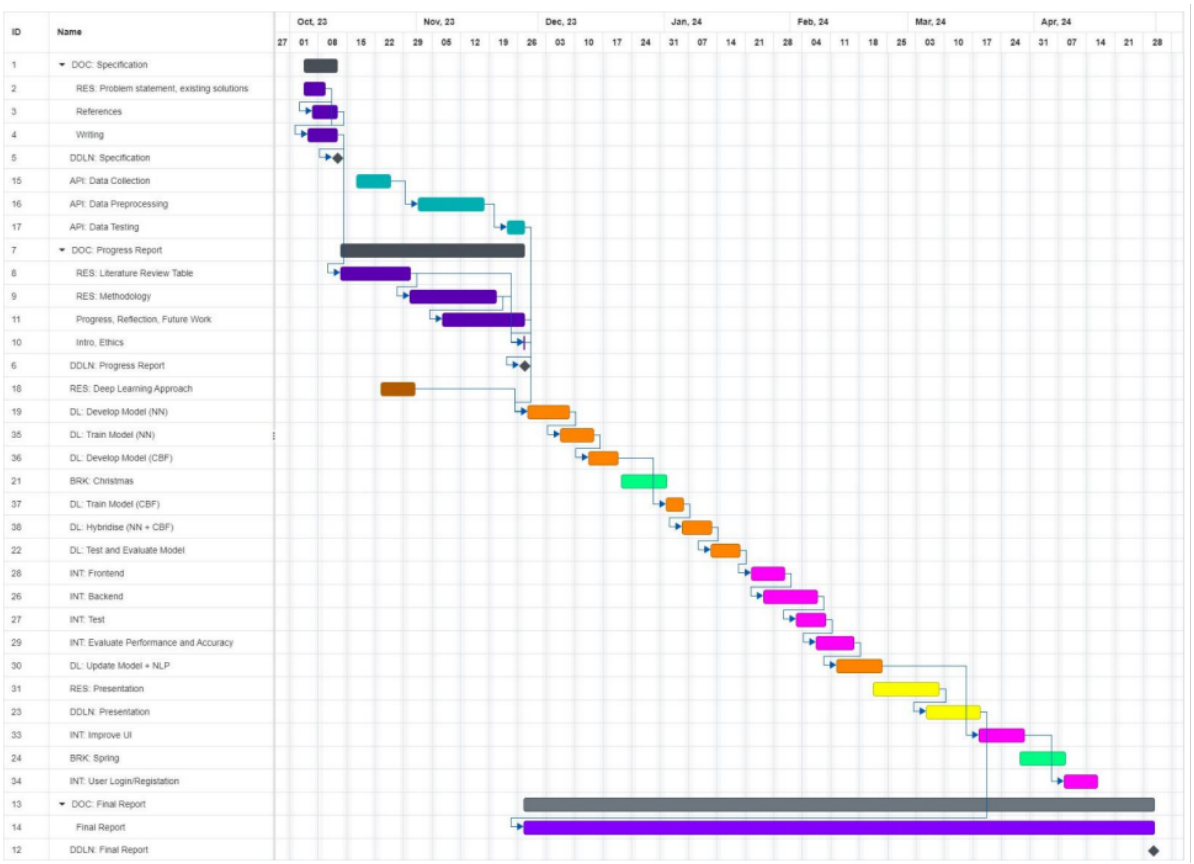


Fig 17: Gantt chart.
DOC – Document, RES – Research,
DDLN – Deadline, API – Application Programming Interface,
DL – Deep Learning, NN – Neural Networks,
CBF – Content-Based Filtering,
BRK – Break, INT – Integration